



UNIVERSITY OF TARTU

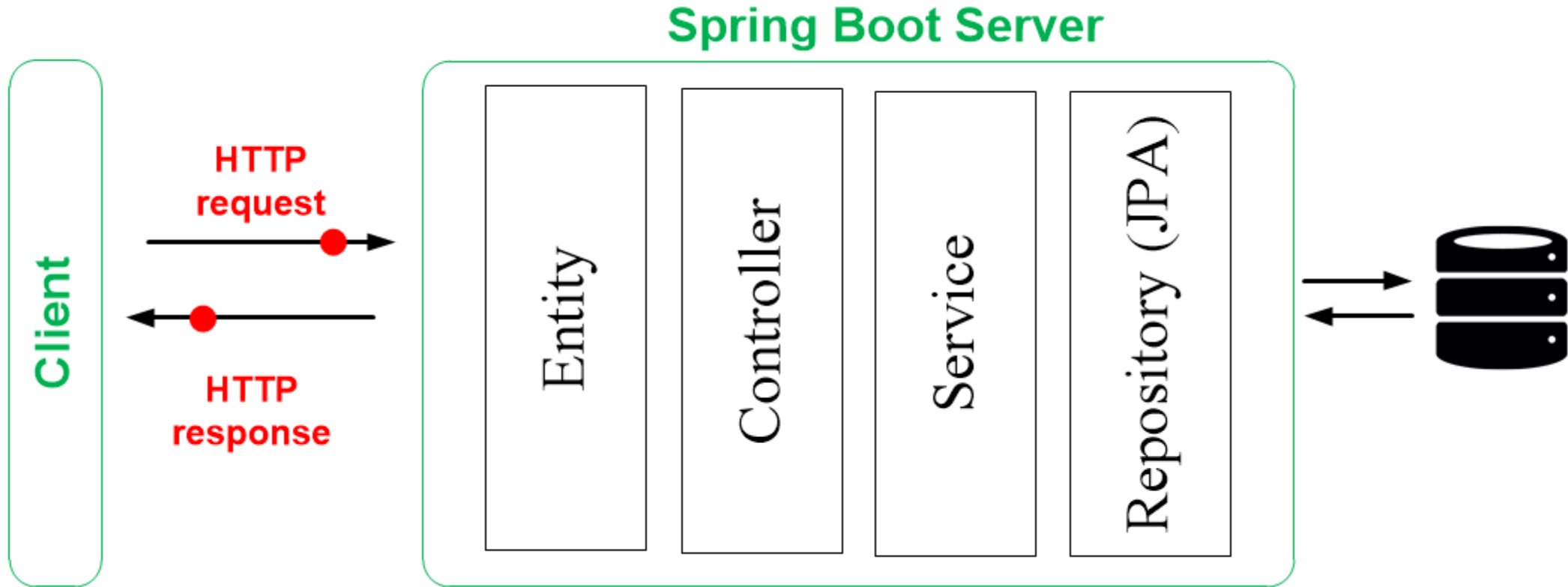


Enterprise System Integration (MTAT.03.229)

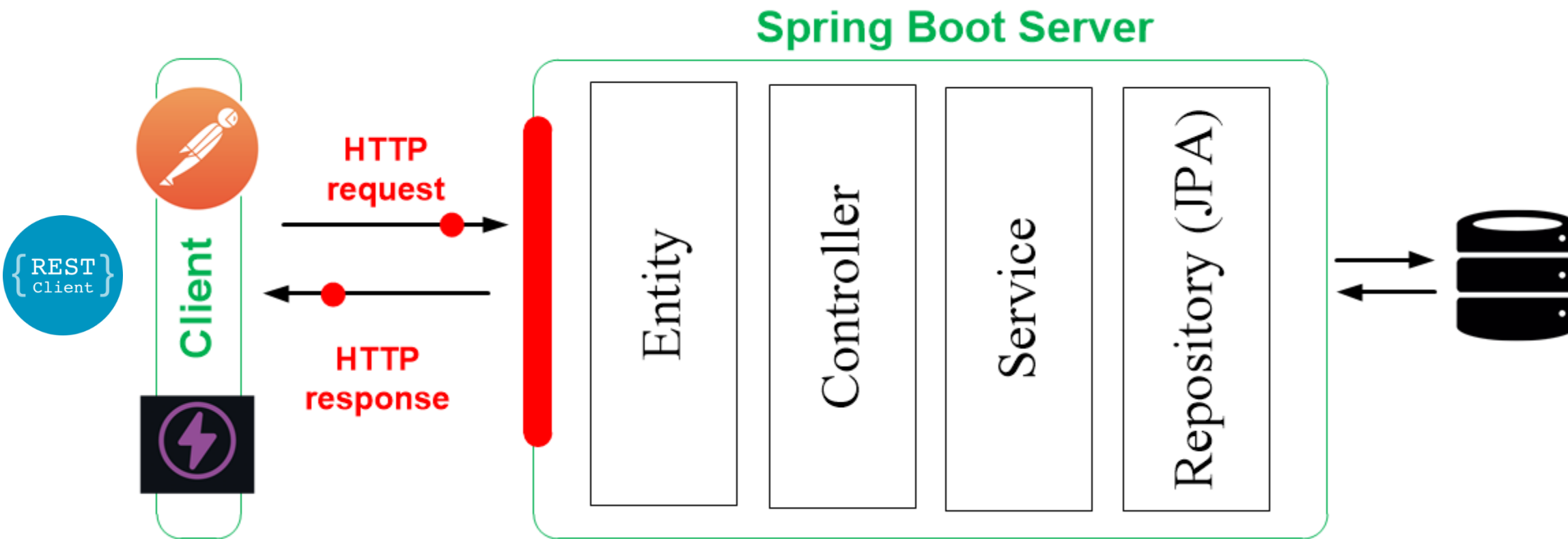
LECTURE 8: VUE.JS

MOHAMAD GHARIB
UNIVERSITY OF TARTU

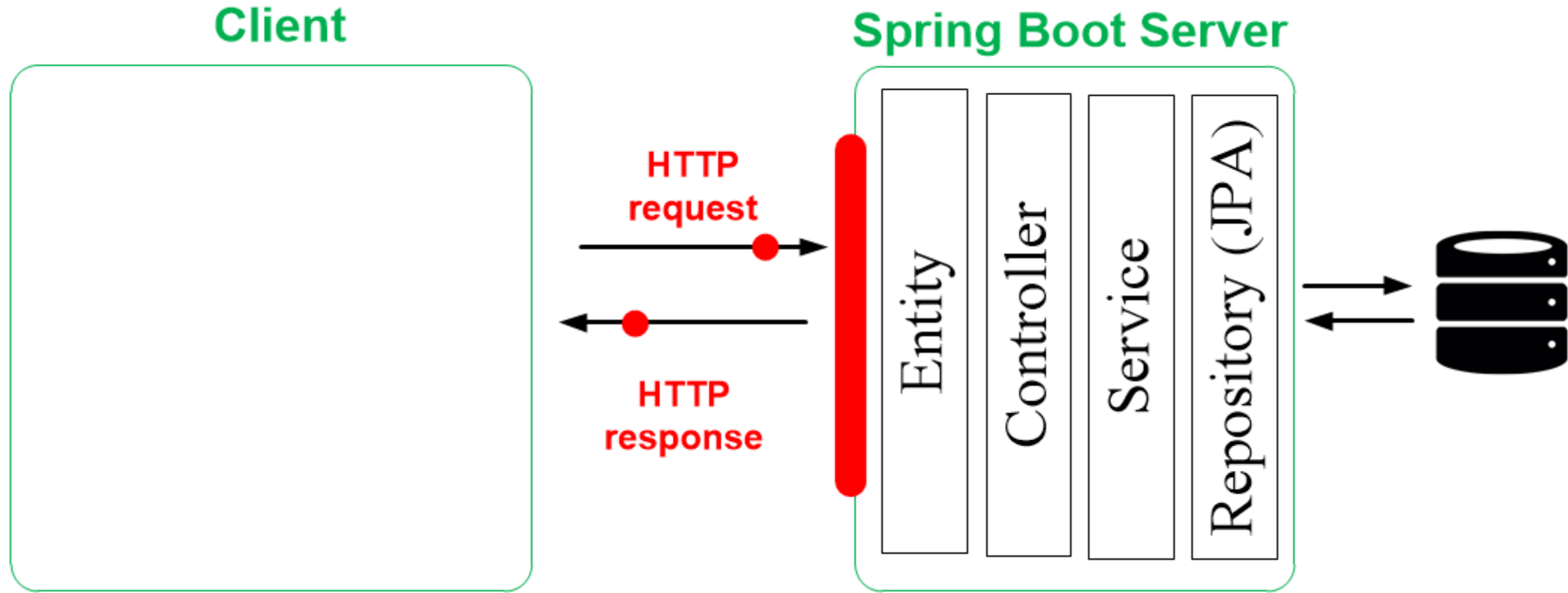
Spring Boot Server - recap



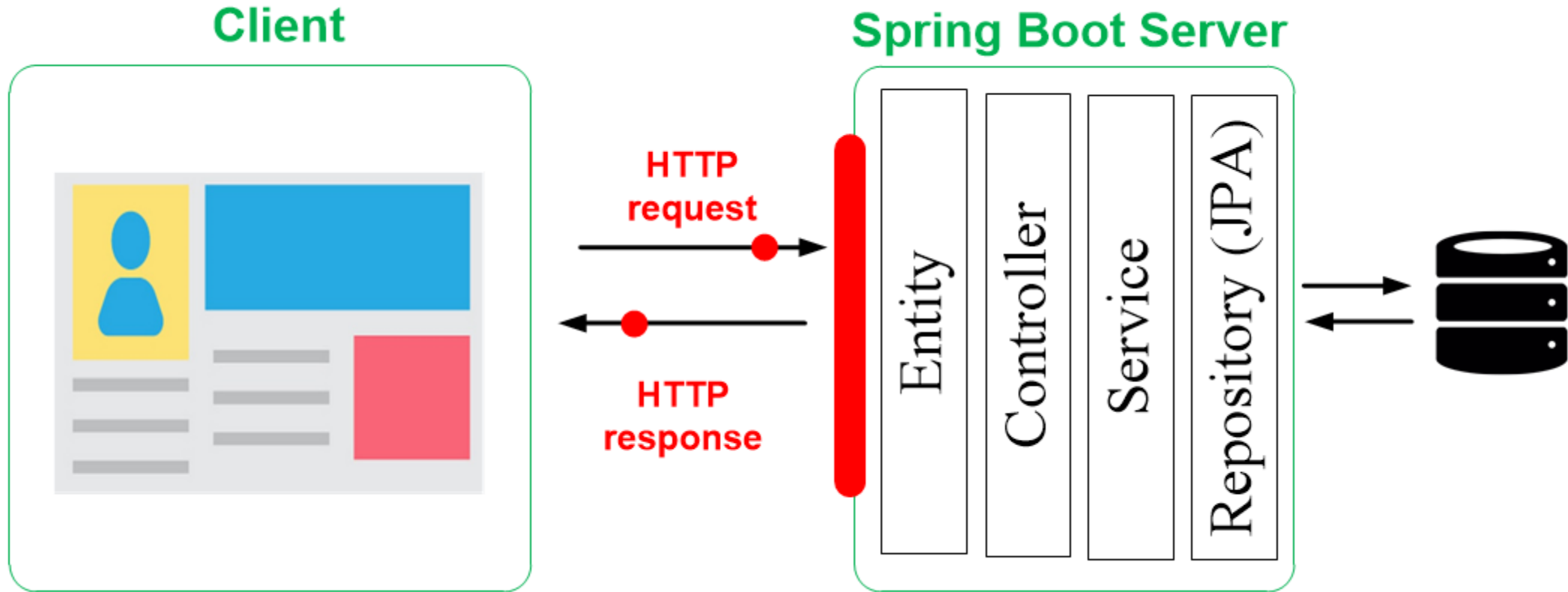
Spring Boot Server - recap



The Client side



The Client side





Vue.js

Vue.js

Vue is a progressive framework for building user interfaces.

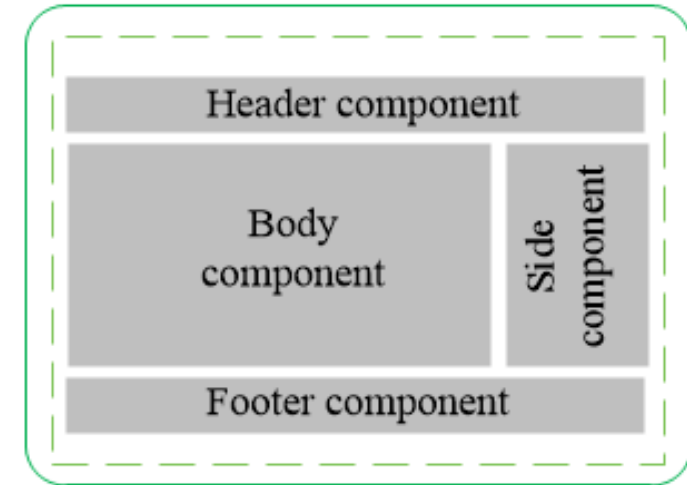
Vue front-end JavaScript framework.

Vue is capable of powering sophisticated Single-Page Applications (SPA), when used in combination with modern tooling and supporting libraries.

Vue can be used also to create Standalone wedges



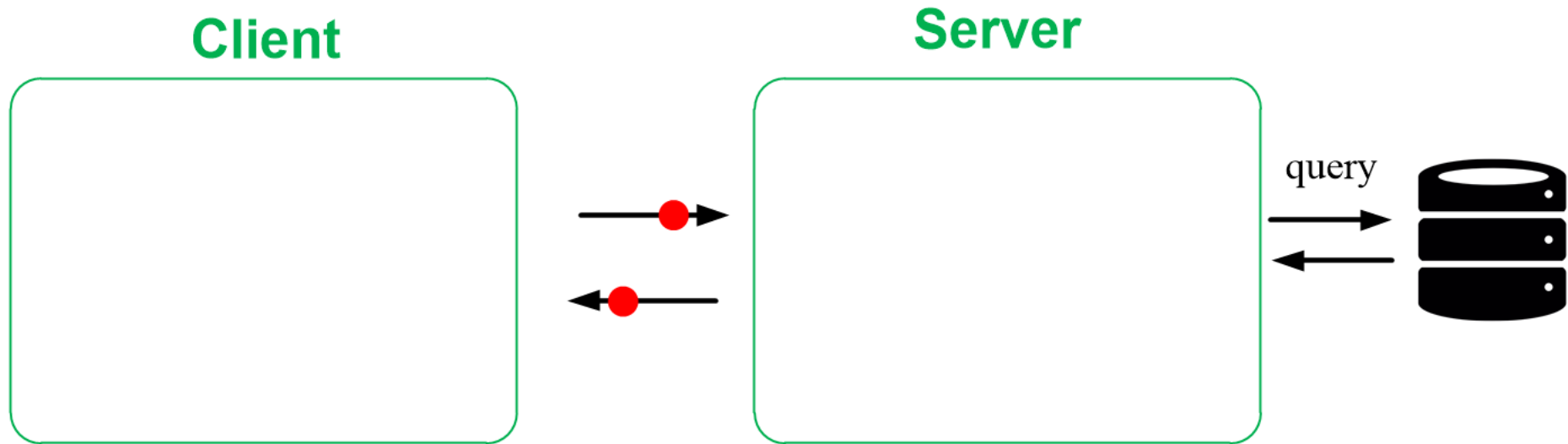
Single-Page Applications (SPA)



- Light weight
- Routing
- Reusable components
- Directives
- Event Handling
- Watched and computed properties
- Unit and Integration testing
- Command Line Interface (CLI) ⁷

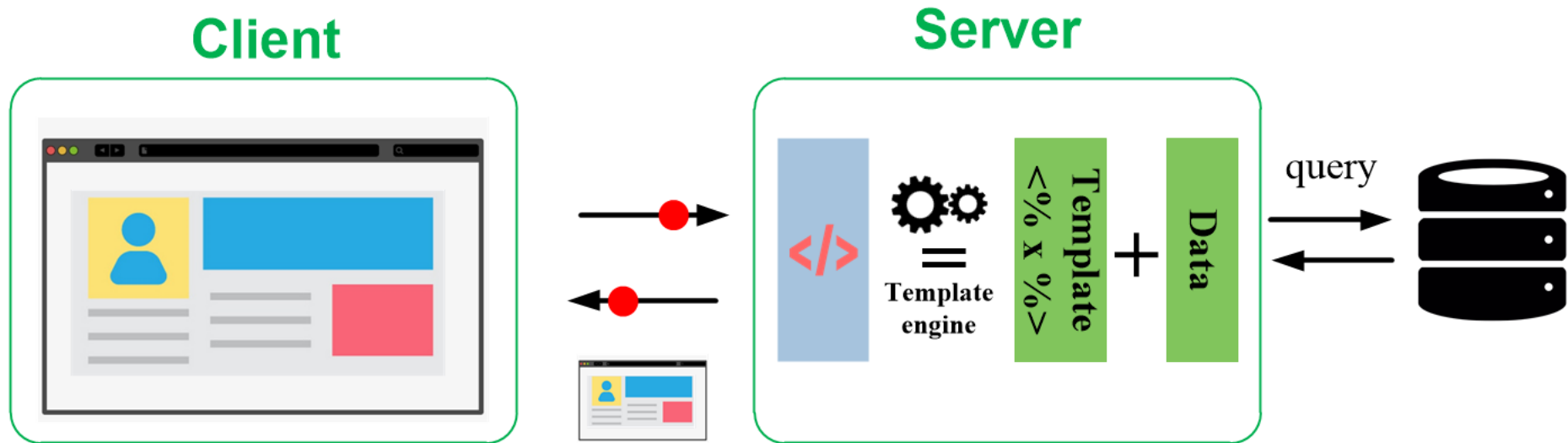
Single-Page Applications (SPA)?

Client-side vs. Server-side rendering



Single-Page Applications (SPA)?

Client-side vs. Server-side rendering

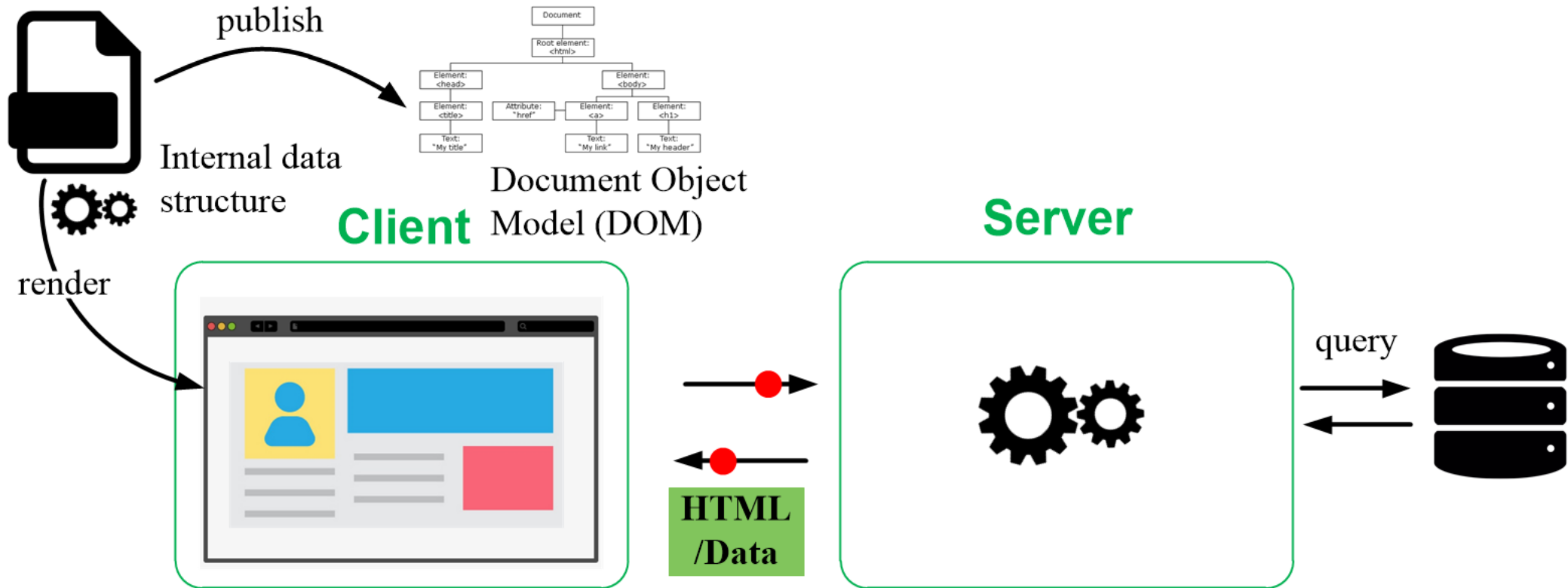


In server-side rendering, the server compiles everything and delivers a fully populated HTML page to the client.

This is done every time you navigated to a new route/page.

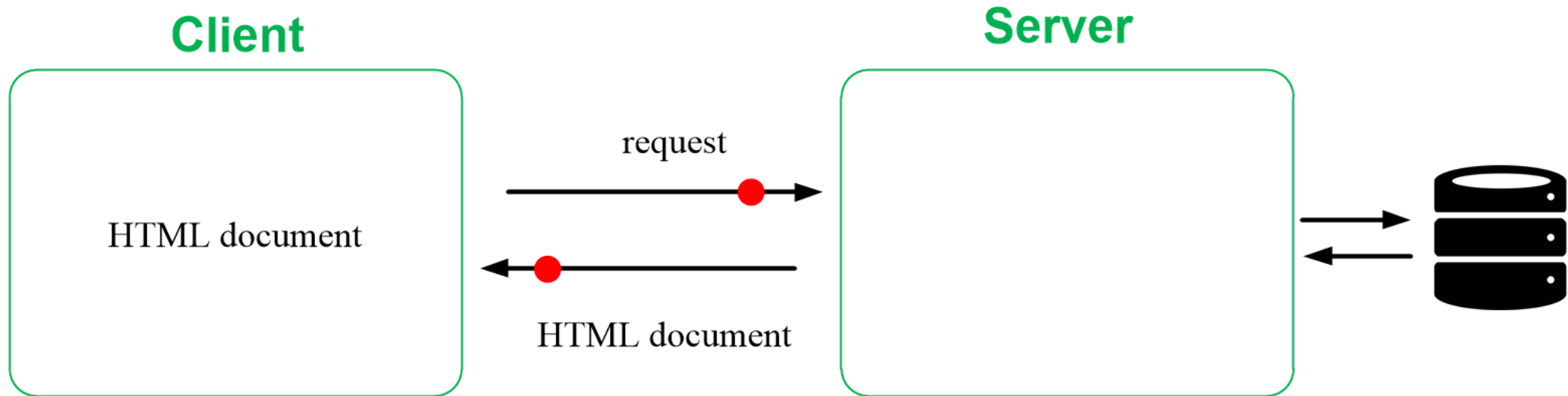
Single-Page Applications (SPA)?

Client-side vs. Server-side rendering

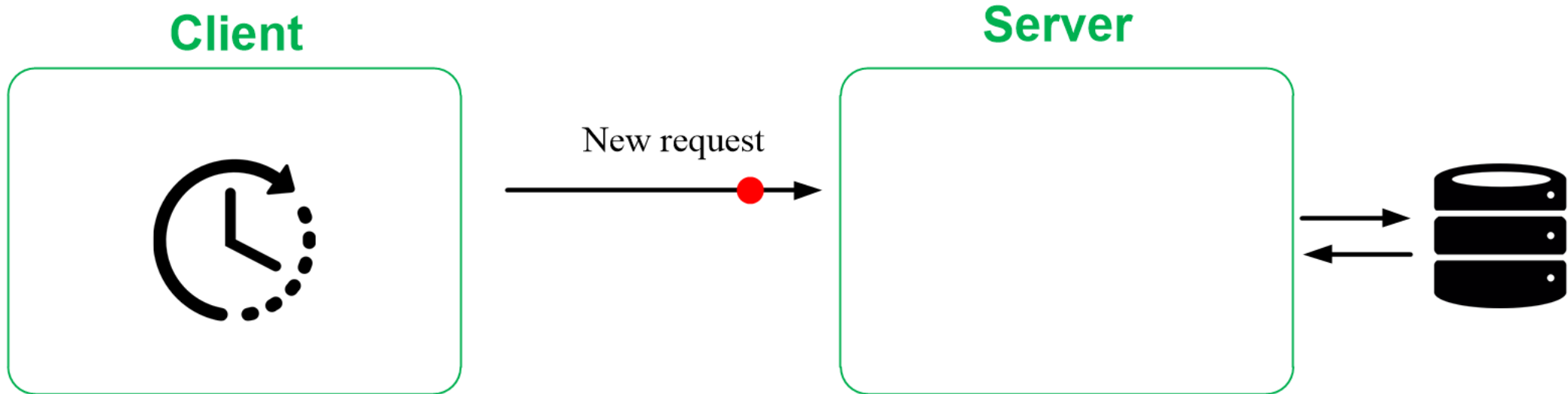


In client-side rendering, the server will deliver a single HTML file without “any content” to the client. Then, the client fetches and compile everything before rendering the content.

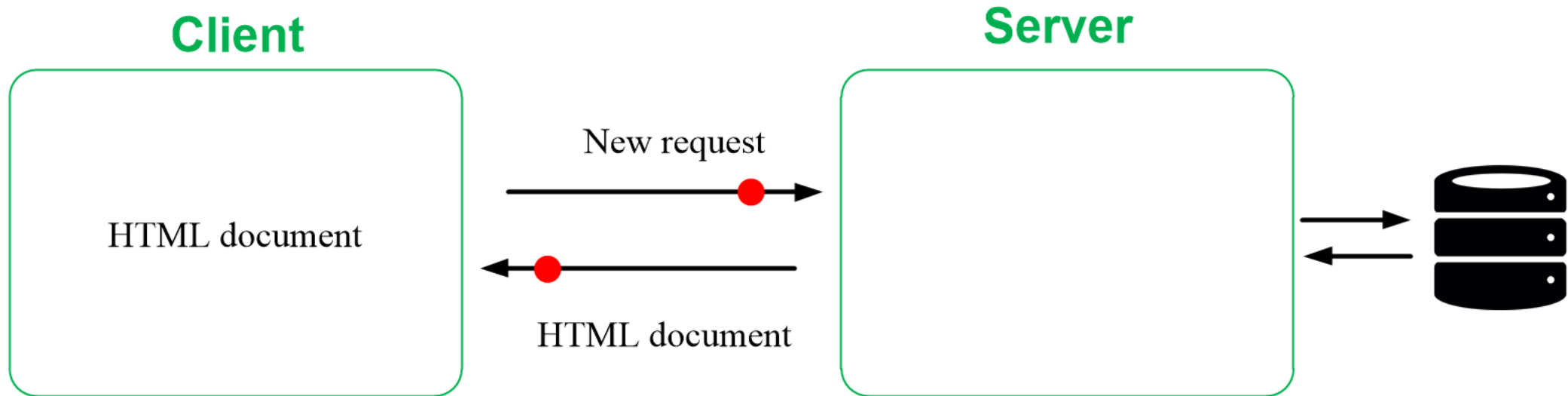
Non Single-Page Applications (SPA)



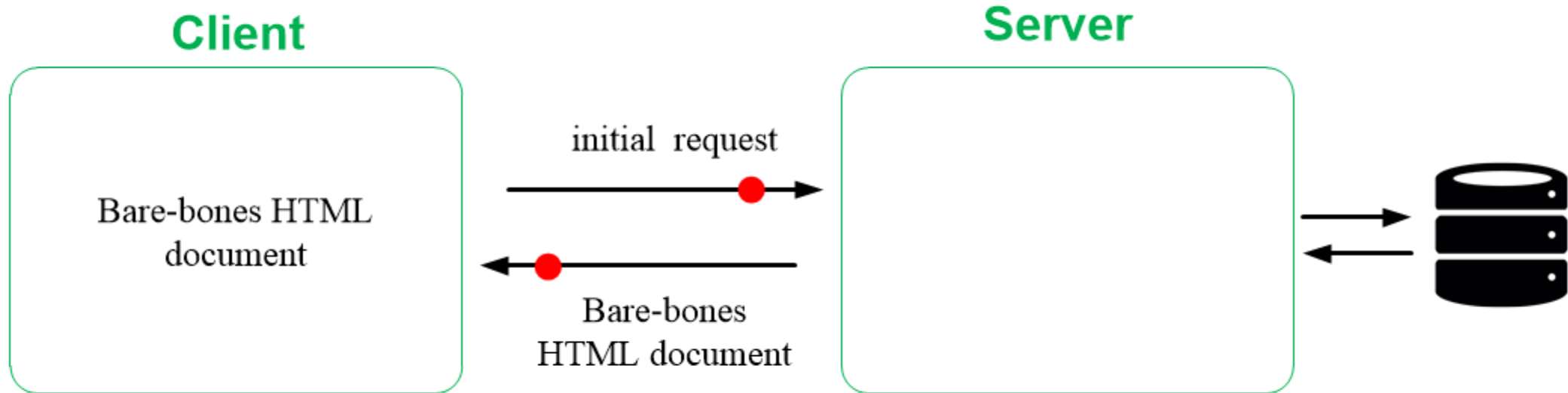
Non Single-Page Applications (SPA)



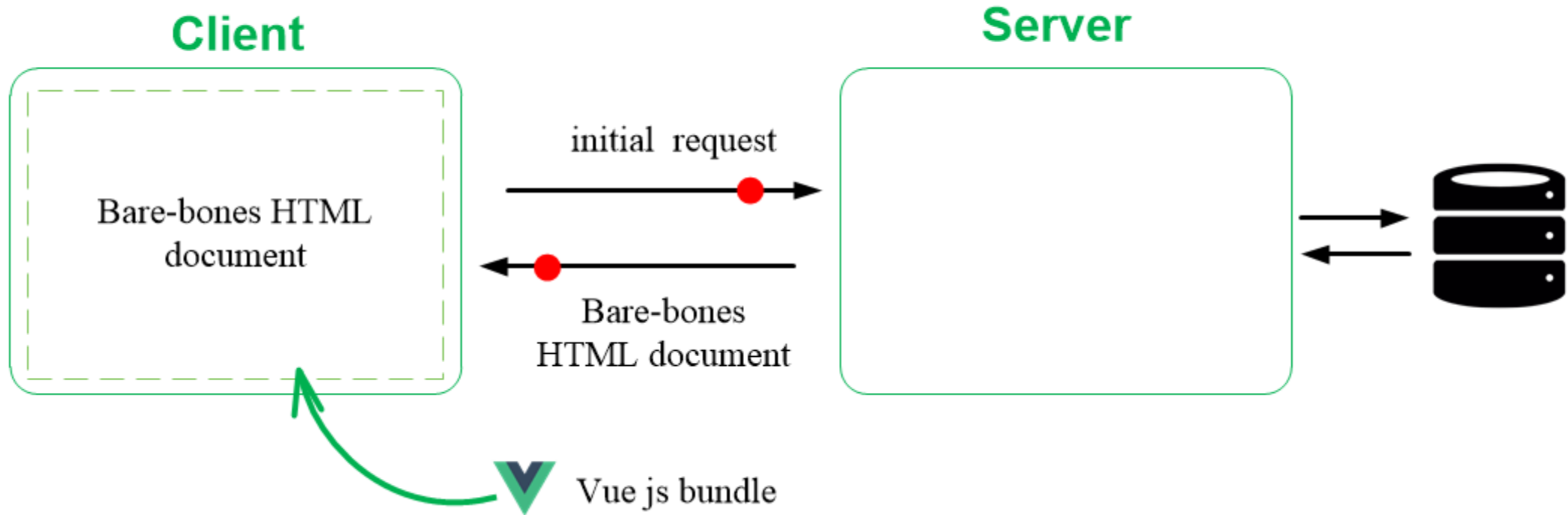
Non Single-Page Applications (SPA)



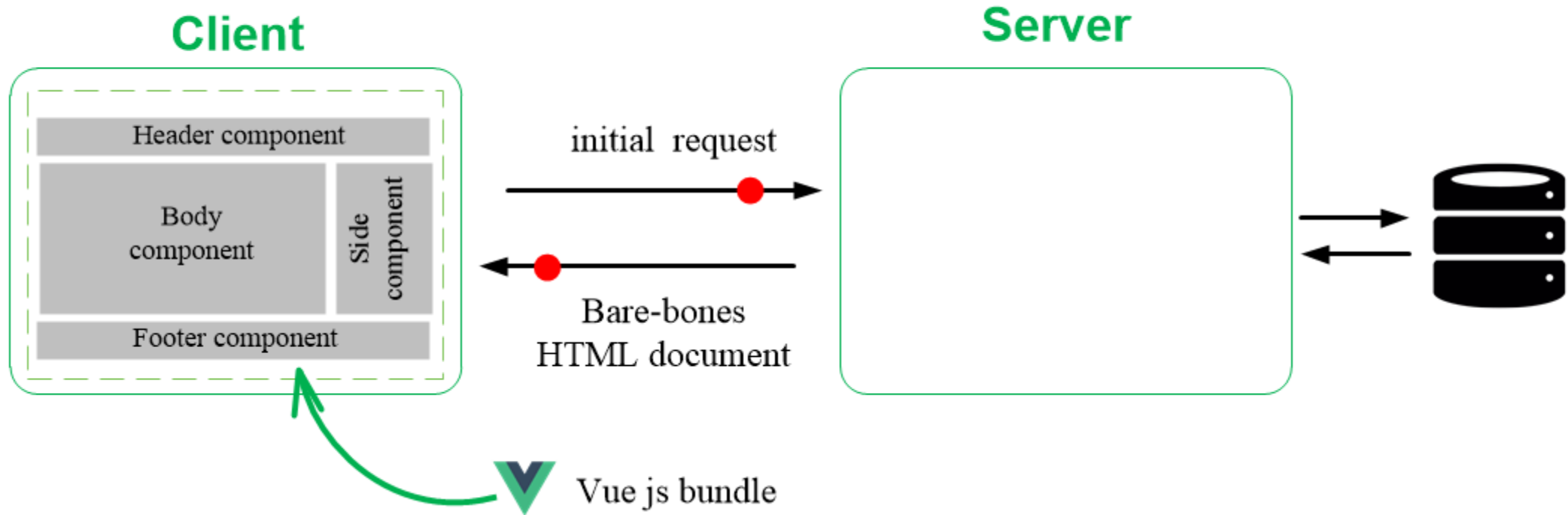
Vue.js - Single-Page Applications (SPA)



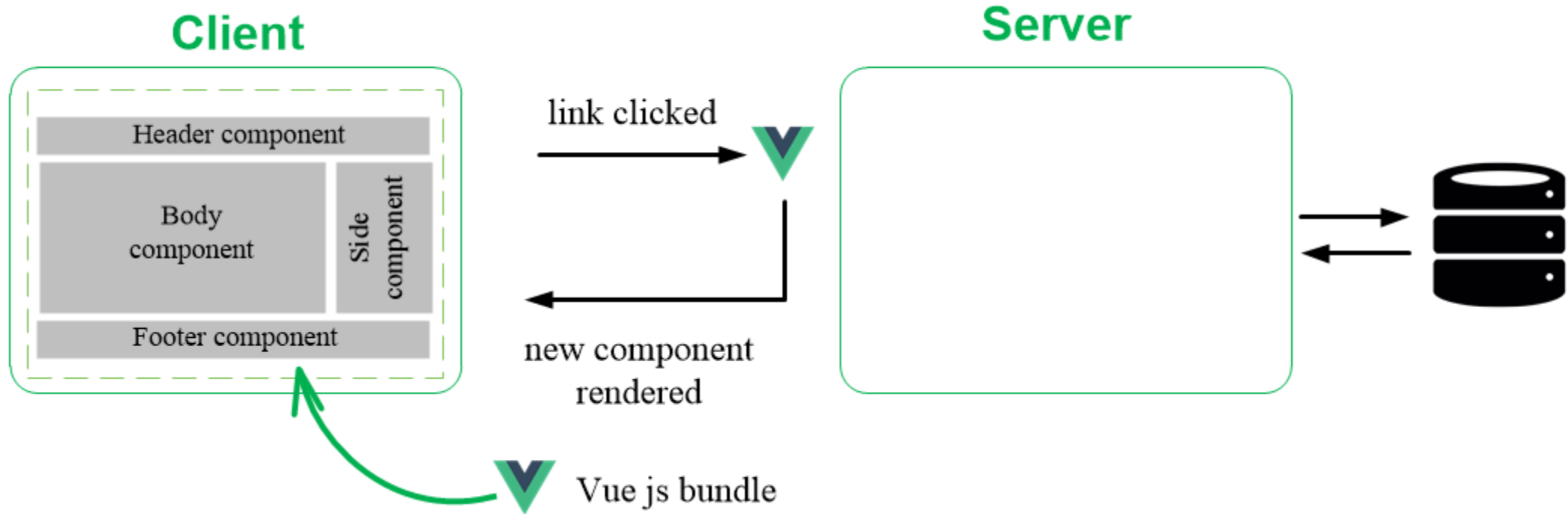
Vue.js - Single-Page Applications (SPA)



Vue.js - Single-Page Applications (SPA)

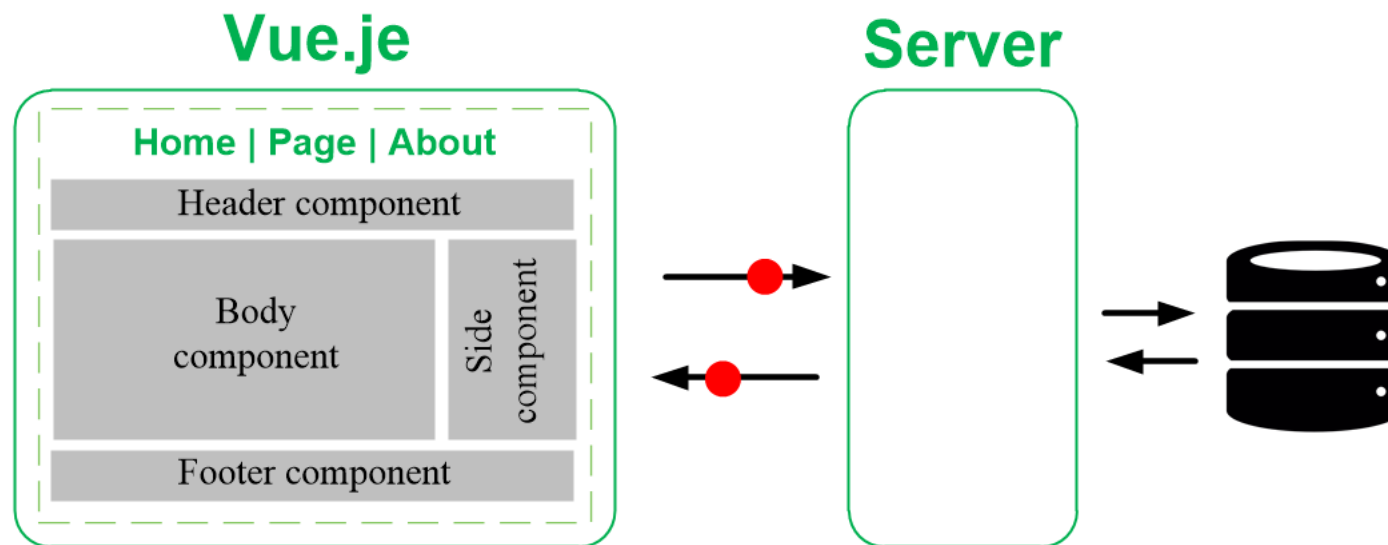


Vue.js - Single-Page Applications (SPA)



Vue.js in nutshell

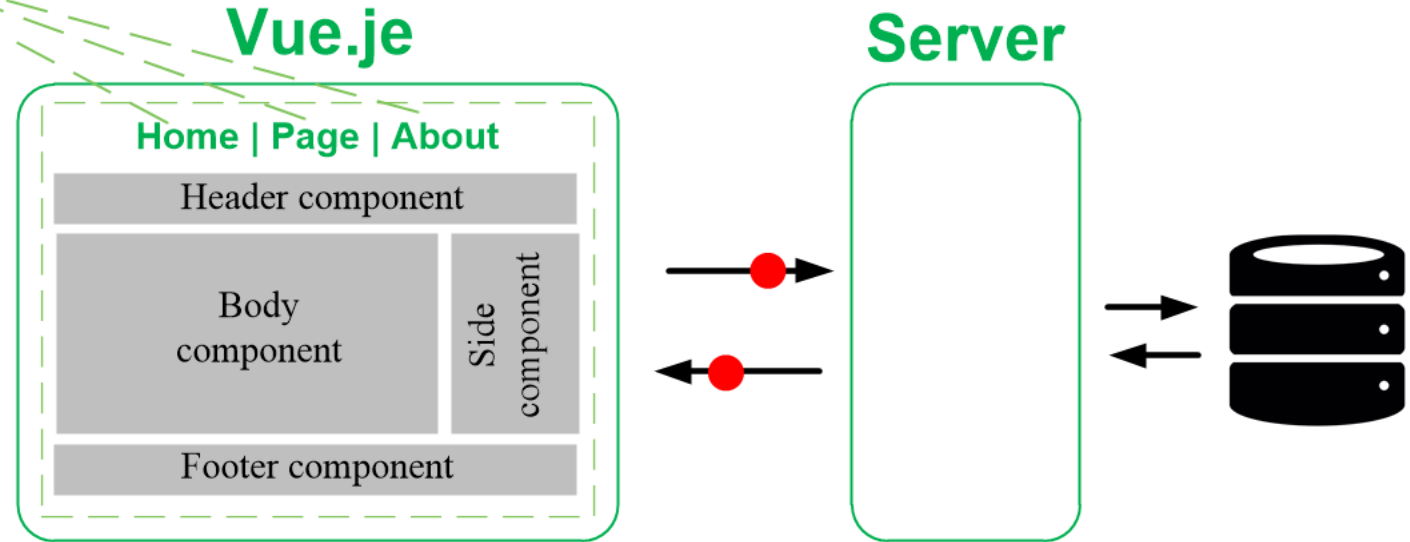
Vue.js in nutshell



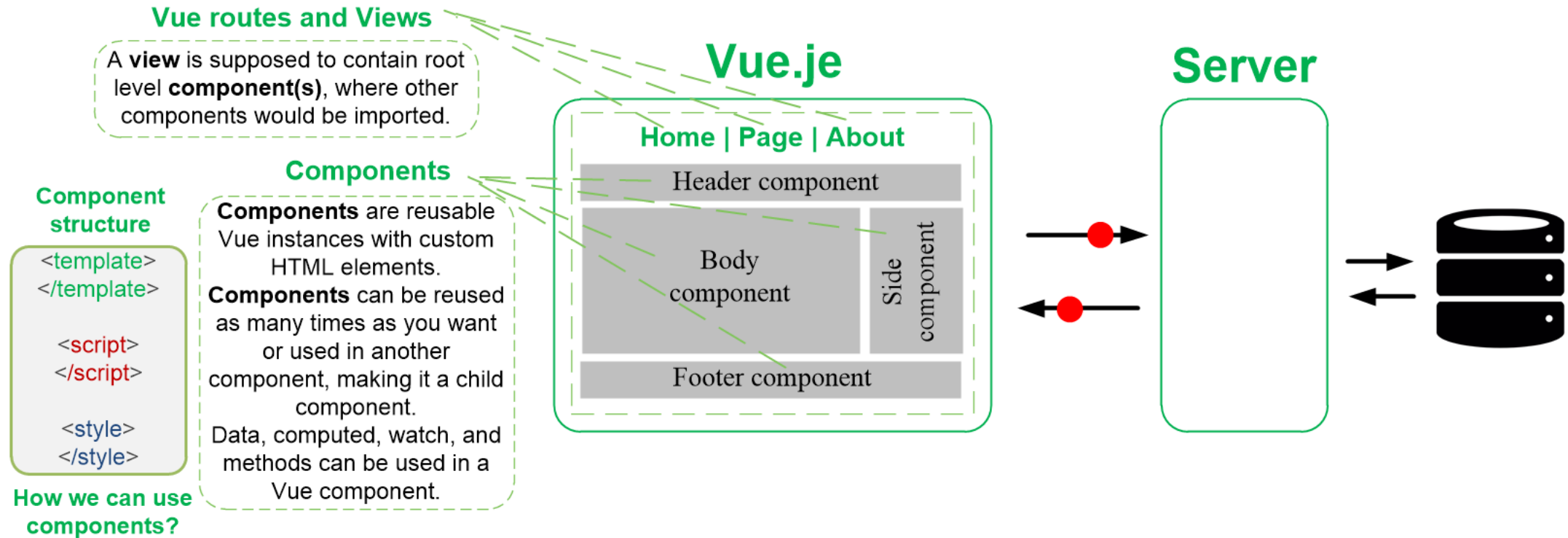
Vue.js in nutshell

Vue routes and Views

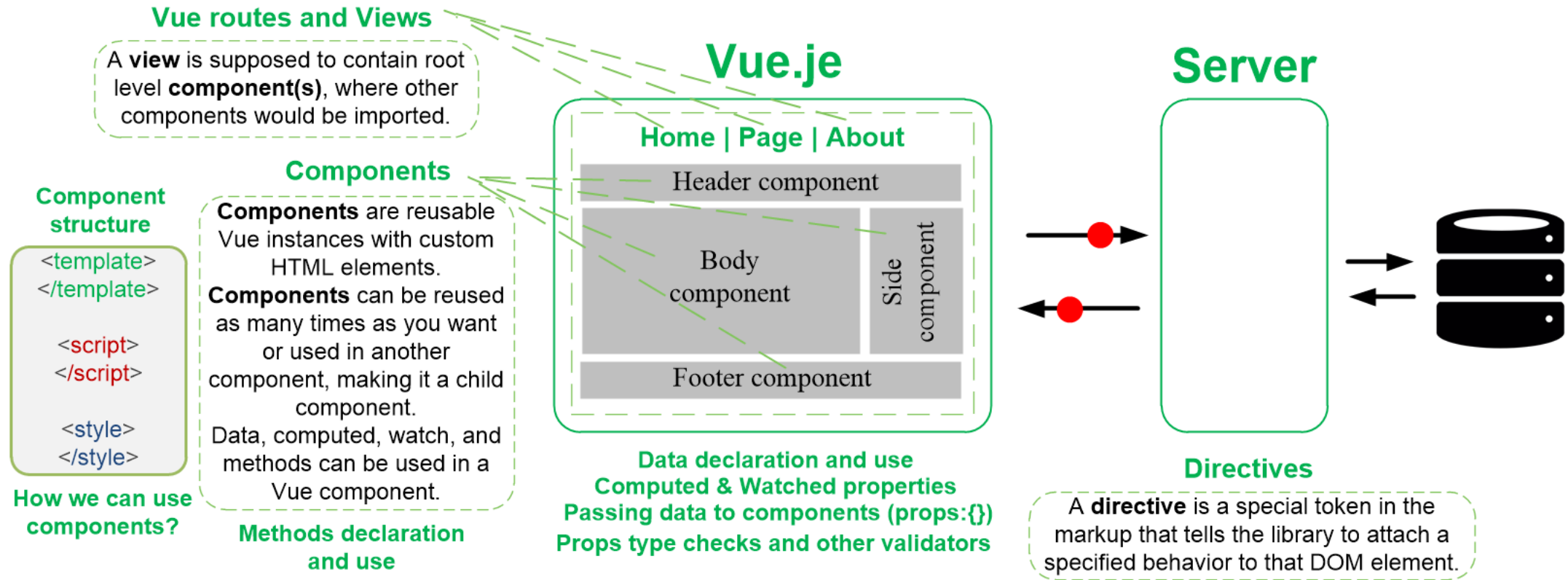
A **view** is supposed to contain root level **component(s)**, where other components would be imported.



Vue.js in nutshell



Vue.js in nutshell



Vue CLI project

Vue CLI project

[Home](#) | [About](#)

The Vue.js logo, a stylized green and blue 'V' shape.

Welcome to Your Vue.js App

For a guide and recipes on how to configure / customize this project, check out the [vue-cli documentation](#).

Installed CLI Plugins

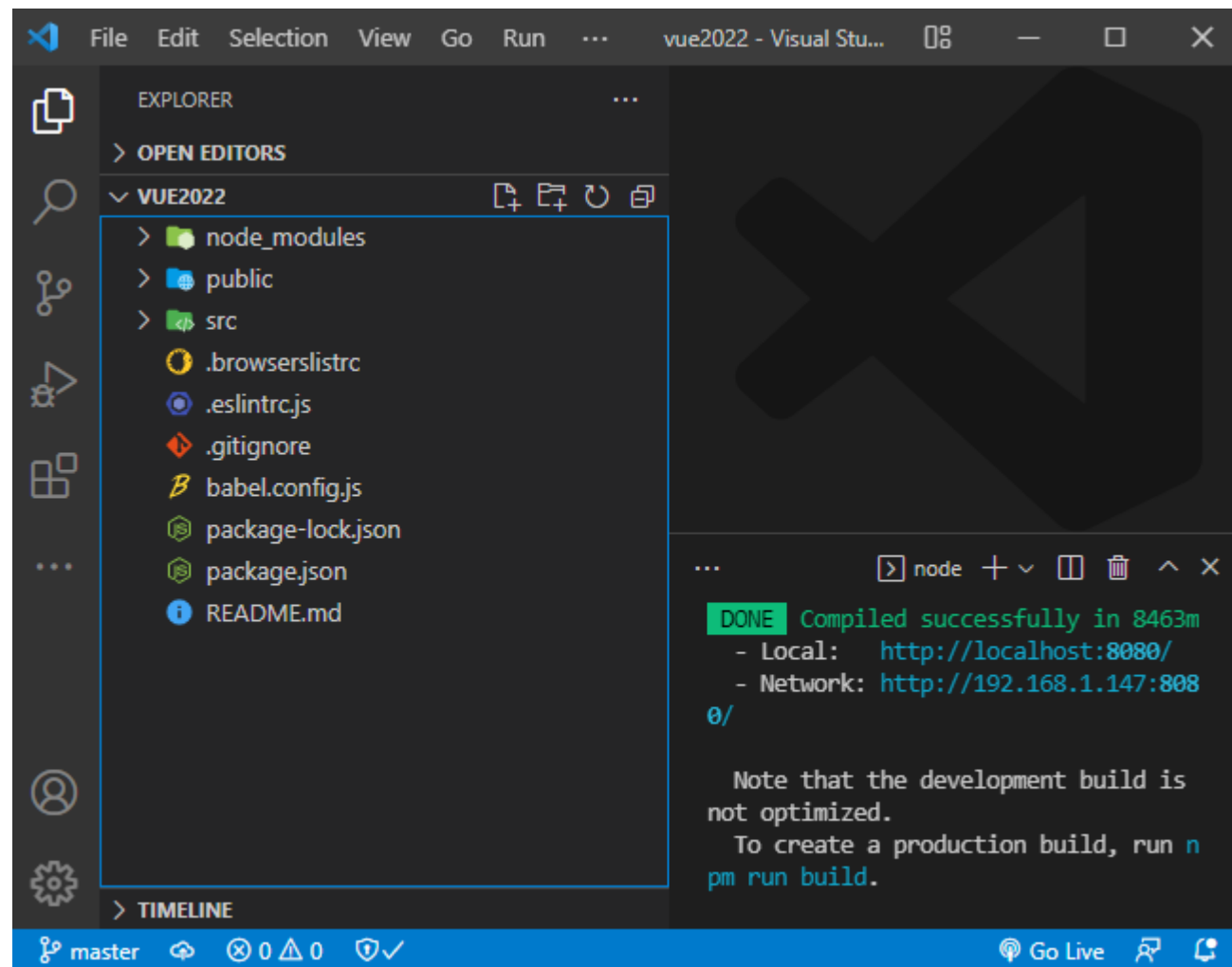
[babel](#) [router](#) [vuex](#) [eslint](#)

Essential Links

[Core Docs](#) [Forum](#) [Community Chat](#) [Twitter](#) [News](#)

Ecosystem

[vue-router](#) [vuex](#) [vue-devtools](#) [vue-loader](#) [awesome-vue](#)



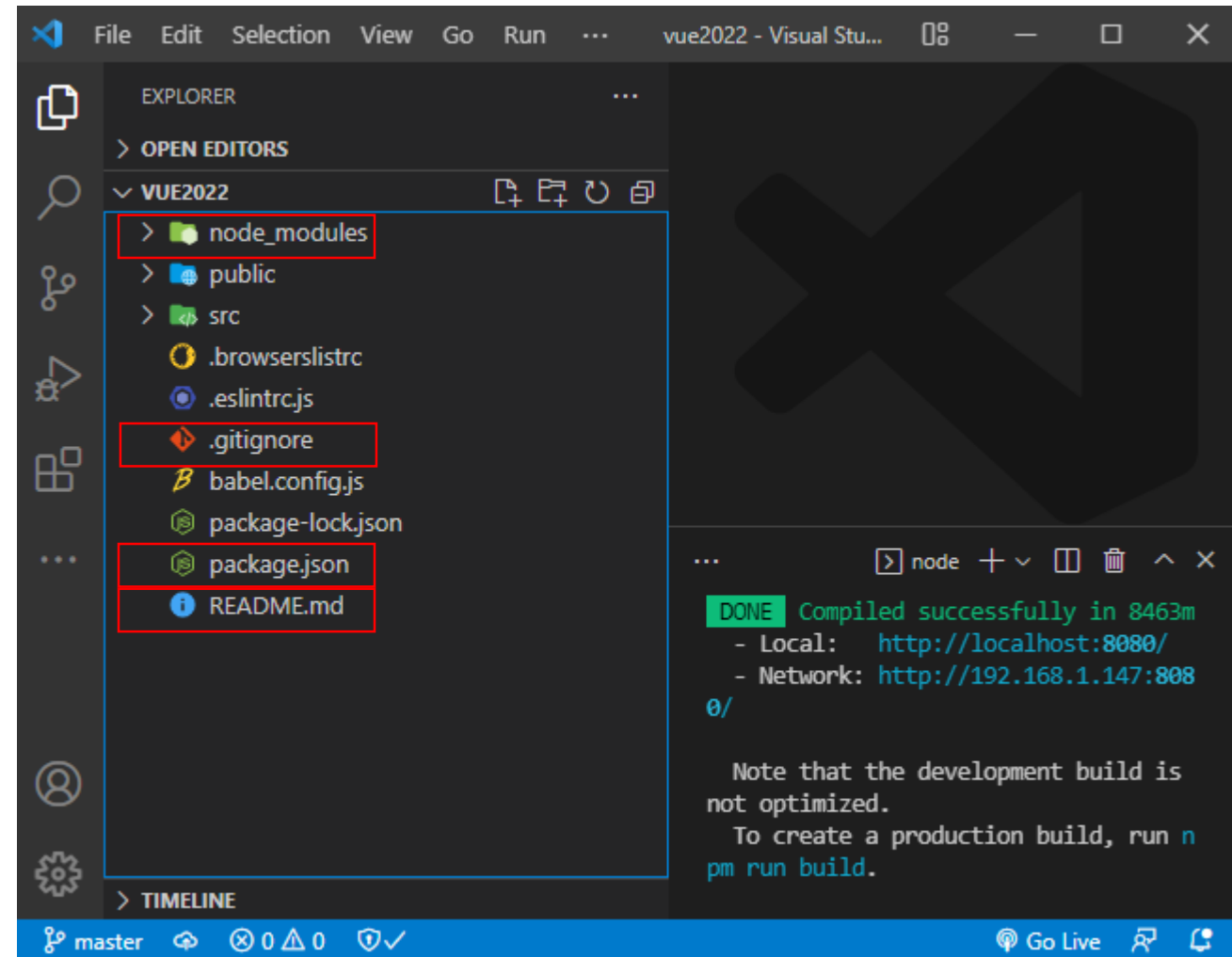
Vue CLI project

node_modules directory contains the full set of dependencies for the application.

package.json file used by the application to know which node_modules to get.

.gitignore contains files that will not be committed to the repo during git push or commit operations.

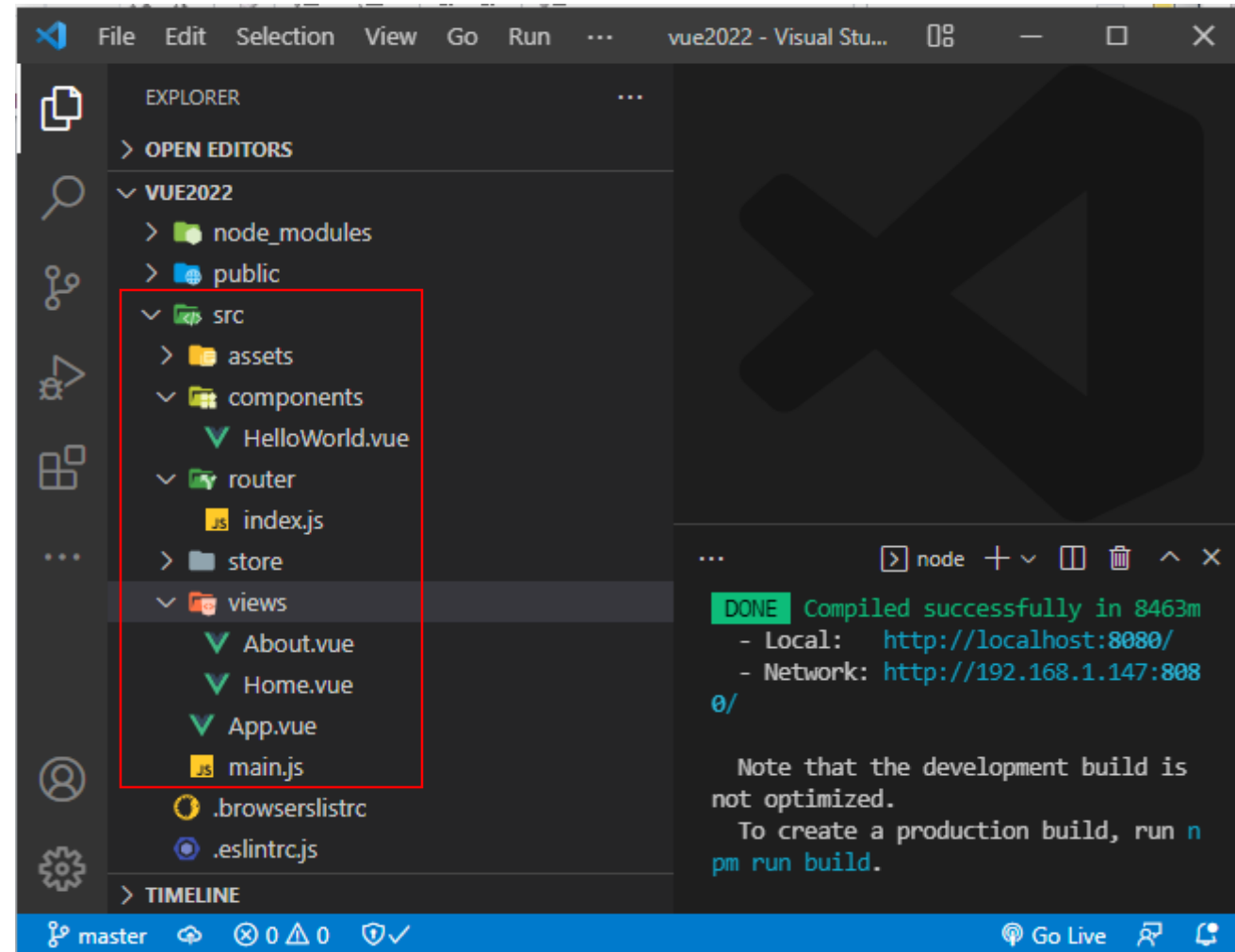
README.md basic readme file that contains essential commands to install and run your project.



Vue CLI project

src directory contains:

- assets
- components
 - HelloWorld.vue
- router
 - index.js
- store
- views
 - About.vue
 - Home.vue
- App.vue
- main.js

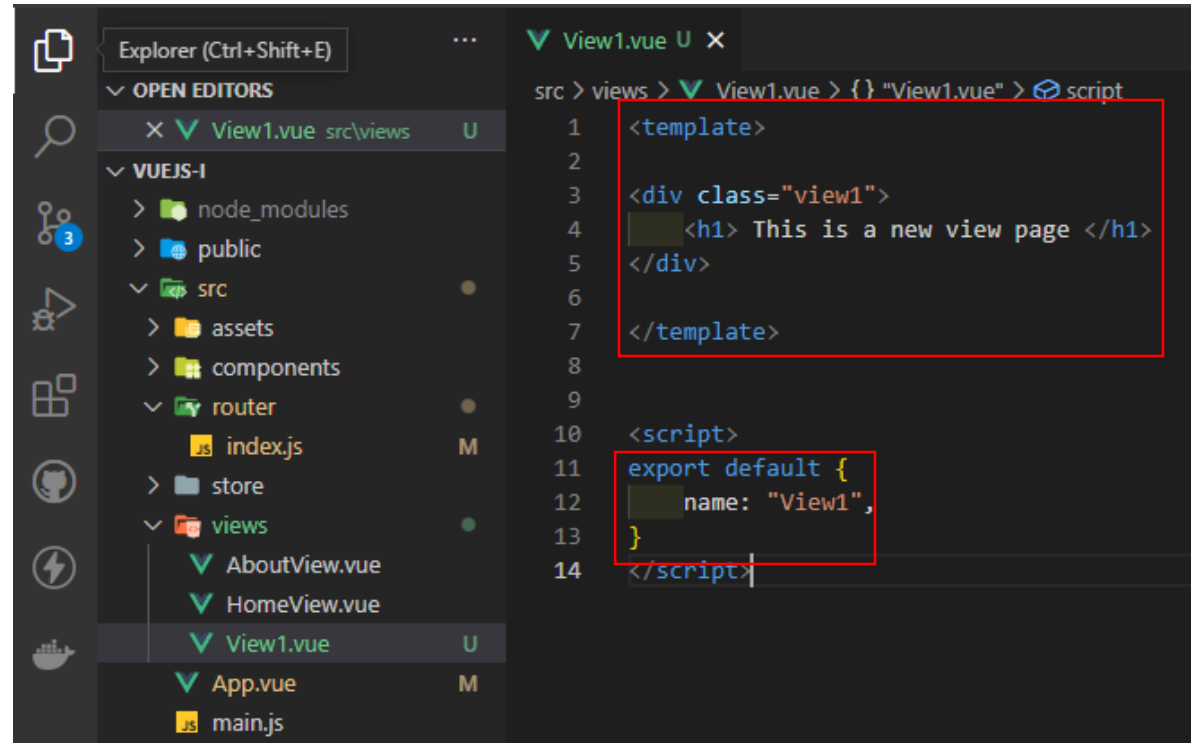


Vue Routers & Views

Views & Vue Router

1. Create a View

- In the Views directory, create a new file (e.g., View1.vue).
- Export the view.

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows the project structure with the 'views' directory expanded, listing 'AboutView.vue', 'HomeView.vue', 'View1.vue', 'App.vue', and 'main.js'. The 'View1.vue' file is selected. The main editor area shows the content of 'View1.vue' with two sections highlighted by red boxes. The first box highlights the template section, which contains a div with a class 'view1' and an h1 element with the text 'This is a new view page'. The second box highlights the script section, which contains an 'export default' statement with a 'name' property set to 'View1'.

```
src > views > View1.vue > {} "View1.vue" > script
1  <template>
2
3  <div class="view1">
4    <h1> This is a new view page </h1>
5  </div>
6
7  </template>
8
9
10 <script>
11 export default {
12   name: "View1",
13 }
14 </script>
```

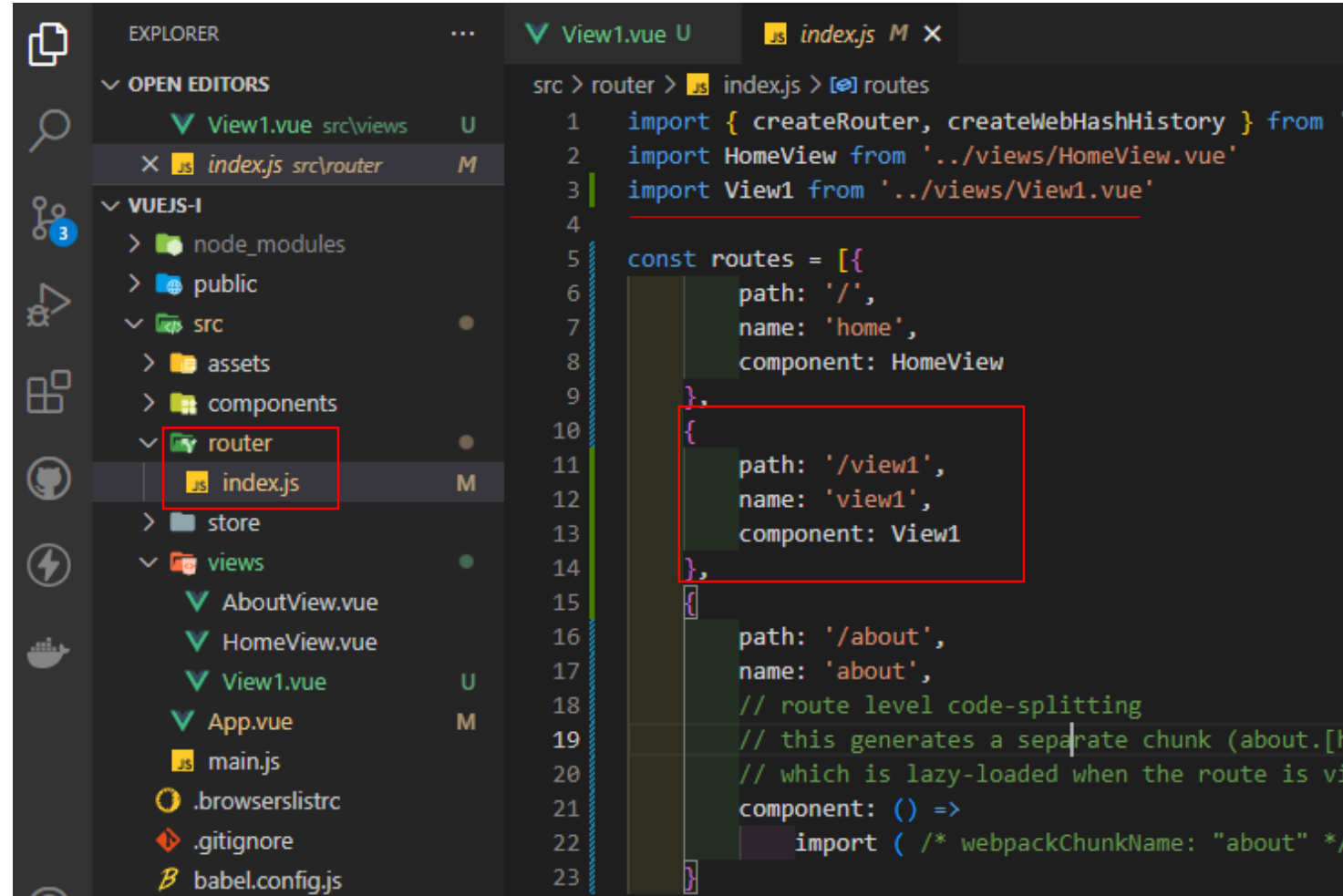
Views & Vue Router

1. Create a View

- In the Views directory, create a new file (e.g., View1.vue).
- Export the view.

2. Create a Vue Route

- In the route/index.js, update the routes array by adding the new route.
- Register/Import the new view.

A screenshot of the Visual Studio Code editor. The Explorer sidebar on the left shows a project structure with a 'views' directory containing 'View1.vue'. The 'router/index.js' file is selected and highlighted with a red box. The main editor area shows the 'index.js' file with the 'routes' array. A new route is being added, highlighted with a red box:

```
const routes = [{
  path: '/',
  name: 'home',
  component: HomeView
}, {
  path: '/view1',
  name: 'view1',
  component: View1
}, {
  path: '/about',
  name: 'about',
  // route level code-splitting
  // this generates a separate chunk (about.[hash].js)
  // which is lazy-loaded when the route is visited
  component: () =>
    import(/* webpackChunkName: "about" */ '@/views/about.vue')
}]
```

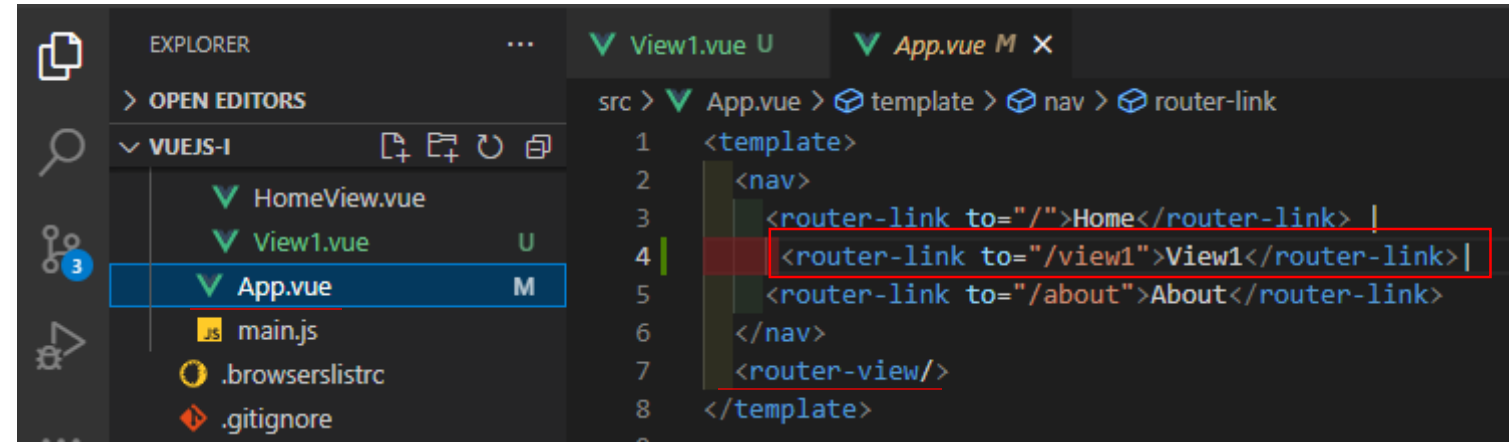
Views & Vue Router

1. Create a View

- In the Views directory, create a new file (e.g., View1.vue).
- Export the view.

2. Create a Vue Route

- In the route/index.js, update the routes array by adding the new route.
- Register/Import the new view.

A screenshot of the Visual Studio Code editor. The Explorer sidebar on the left shows a project named 'VUEJS-I' with files 'HomeView.vue', 'View1.vue', and 'App.vue'. 'App.vue' is selected. The main editor area shows the 'App.vue' file with the 'template' tab active. The code is as follows:

```
1 <template>
2   <nav>
3     <router-link to="/">Home</router-link> |
4     <router-link to="/view1">View1</router-link> |
5     <router-link to="/about">About</router-link>
6   </nav>
7   <router-view/>
8 </template>
```

The line for the new route, '`<router-link to="/view1">View1</router-link>`', is highlighted with a red box.

3. Create the router-link

- In App.vue, add the **route link**.

`<router-view />` is a functional component that renders the matched component for the given path.

Views & Vue Router

1. Create a View

- In the Views directory, create a new file (e.g., View1.vue).
- Export the view.

2. Create a Vue Route

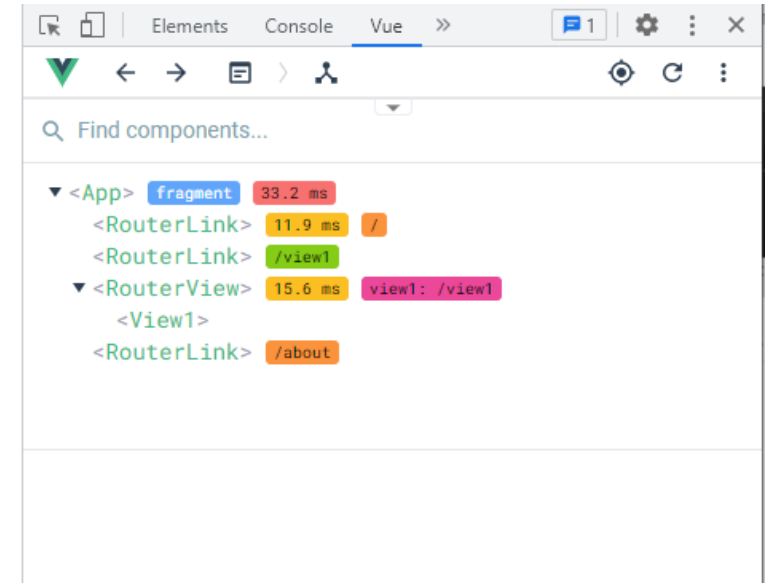
- In the route/index.js, update the routes array by adding the new route.
- Register/Import the new view.

3. Create the router-link

- In App.vue, add the **route link**.

[Home](#) | [View1](#) | [About](#)

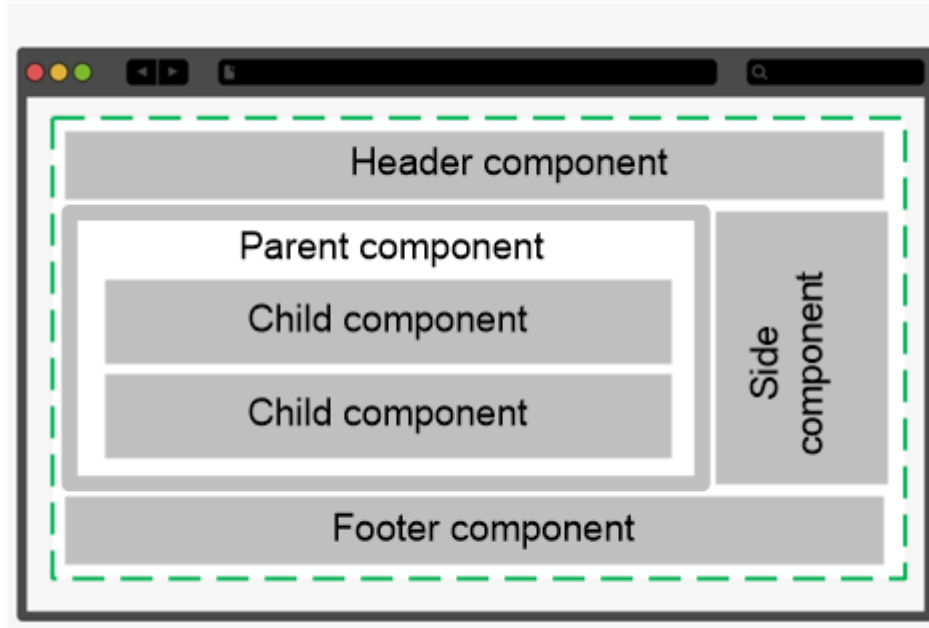
This is a new view page



Components

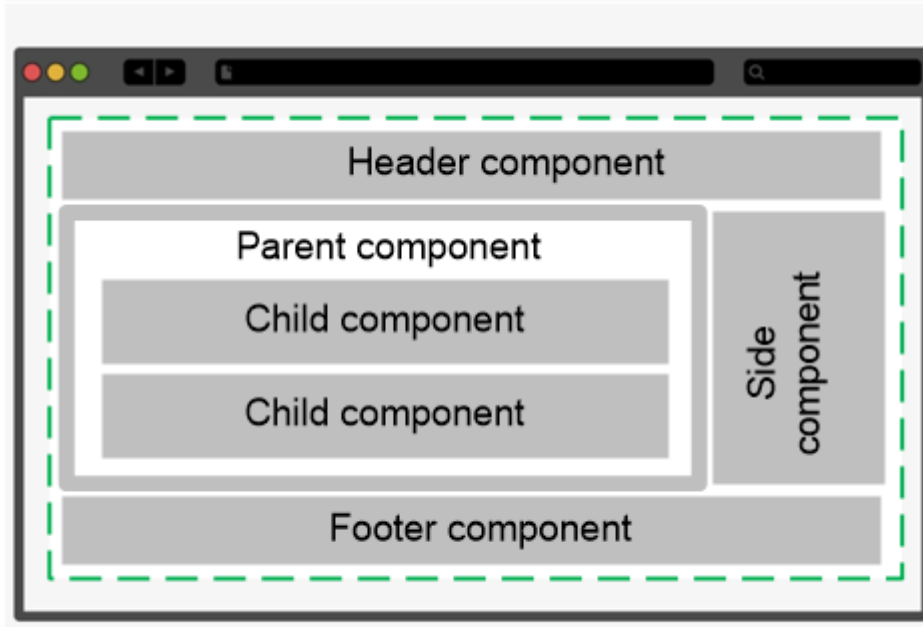
Single File Components (SFC)

Components

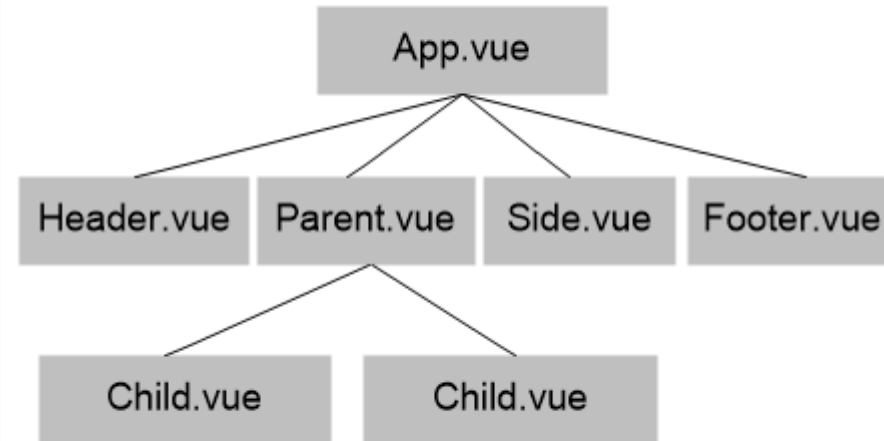


✓ Single-Page Applications (SPA)

Components



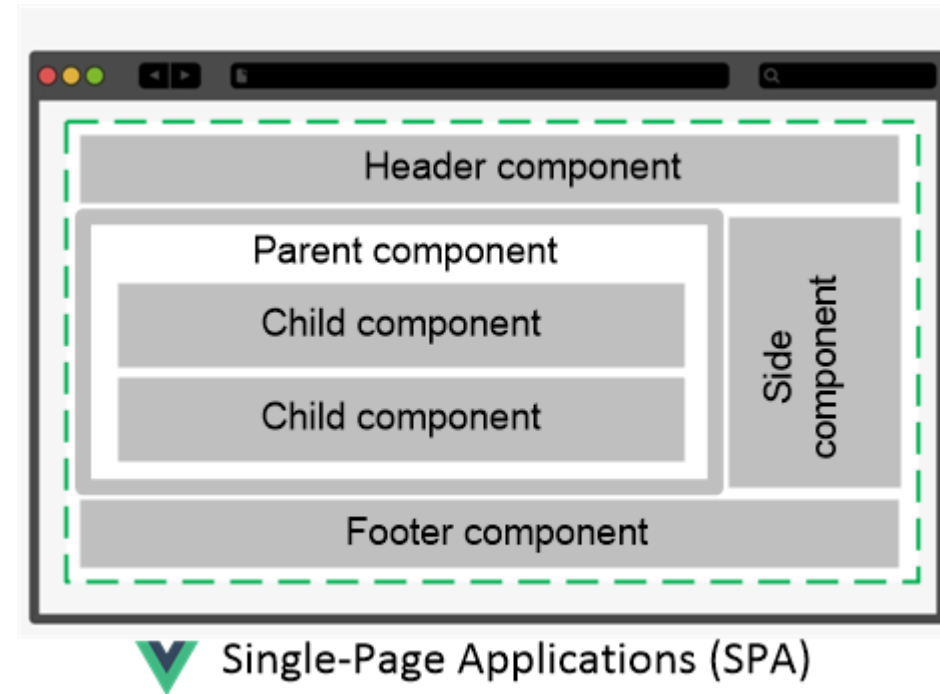
✓ Single-Page Applications (SPA)



Components

Components (SFC) are reusable Vue instances with a name.

We can use this component as a custom element inside a root Vue instance.



Components

Components (SFC) are reusable Vue instances with a name.

We can use this component as a custom element inside a root Vue instance.

A **Vue** component is a special file format that allows us to encapsulate the **template**, **logic**, and **styling** in a single file.

`<template>`

`<HTML>`

`</template>`

`<script>`

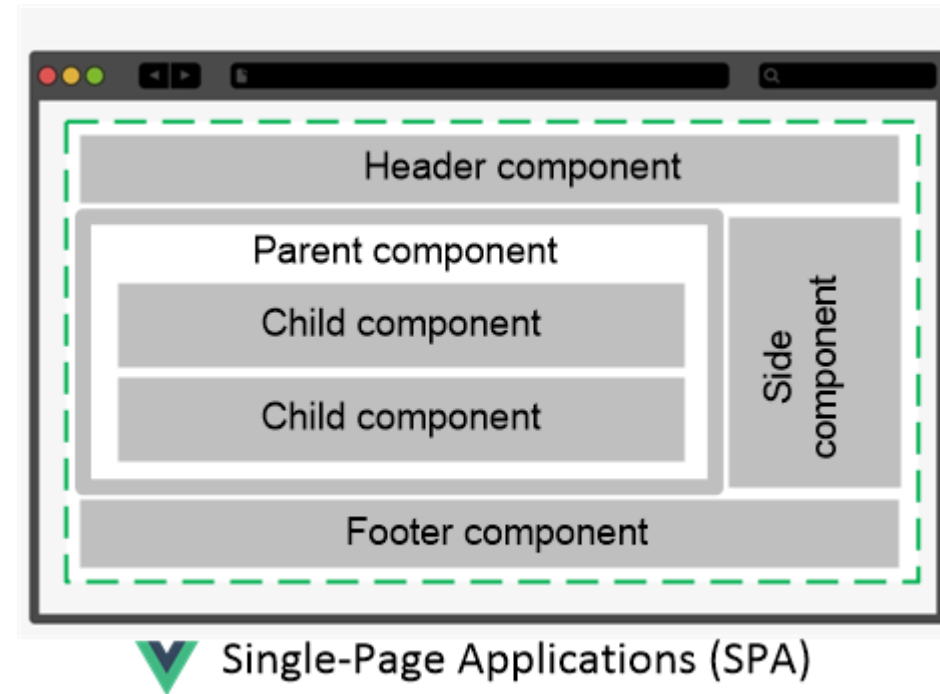
`JavaScript`

`</script>`

`<style>`

`CSS`

`</style>`



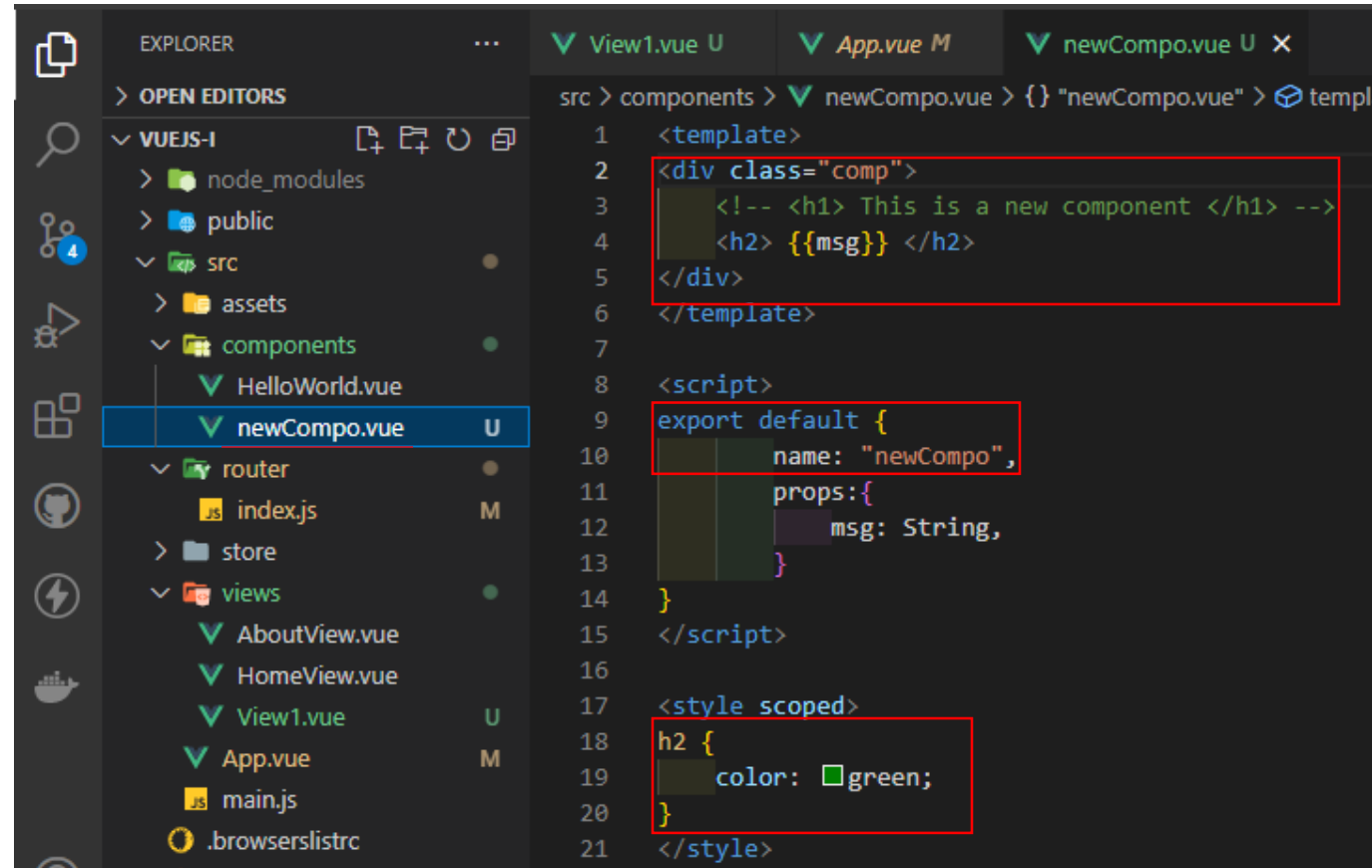
Components

1. Create a component

- In **components** directory, create a new file (e.g., newCompo.vue).
- Add the structure **<template>**, **<script>**, and **<style>**.

2. Export the **component**, i.e., make it available to be used by other **components** and **views**.

export default {name: "newCompo", }

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project structure with a 'components' directory containing 'newCompo.vue', which is selected. The main editor area shows the content of 'newCompo.vue' with three sections highlighted by red boxes: the template section containing HTML for a component with a class 'comp' and a message prop; the script section containing a default export with the component's name and props; and the style section containing a scoped CSS rule for the 'h2' element.

```
1 <template>
2   <div class="comp">
3     <!-- <h1> This is a new component </h1> -->
4     <h2> {{msg}} </h2>
5   </div>
6 </template>
7
8 <script>
9   export default {
10     name: "newCompo",
11     props: {
12       msg: String,
13     }
14   }
15 </script>
16
17 <style scoped>
18   h2 {
19     color: green;
20   }
21 </style>
```

Components

1. Create a component

- In **components** directory, create a new file (e.g., newCompo.vue).
- Add the structure **<template>**, **<script>**, and **<style>**.

2. Export the **component**, i.e., make it available to be used by other **components** and **views**.

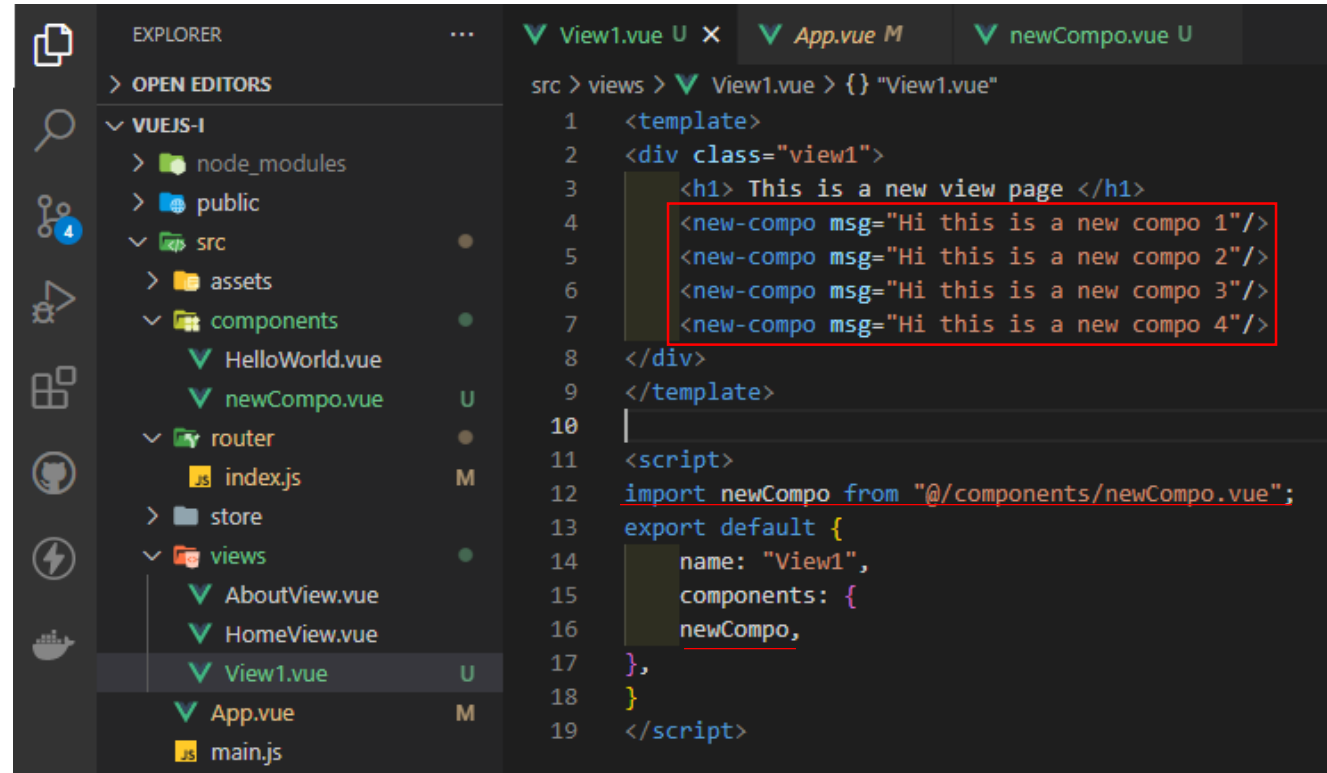
export default {name: "newCompo", }

3. Register/Import the new component in the using **component/view**.

4. The newly created **component** can be used in another **component** or **view** as a **tag** using the key it is registered under.

<newCompo />

<new-compo> </new-compo>



The screenshot shows a VS Code editor with a Vue.js project. The Explorer panel on the left shows the project structure: VUEJS-I, node_modules, public, src (assets, components, router, store, views), and main.js. The components directory contains HelloWorld.vue and newCompo.vue. The views directory contains AboutView.vue, HomeView.vue, and View1.vue. The App.vue file is also visible. The View1.vue file is open in the editor, showing the following code:

```
src > views > View1.vue > {} "View1.vue"
1  <template>
2  <div class="view1">
3      <h1> This is a new view page </h1>
4      <new-compo msg="Hi this is a new compo 1"/>
5      <new-compo msg="Hi this is a new compo 2"/>
6      <new-compo msg="Hi this is a new compo 3"/>
7      <new-compo msg="Hi this is a new compo 4"/>
8  </div>
9  </template>
10
11 <script>
12 import newCompo from "@components/newCompo.vue";
13 export default {
14     name: "View1",
15     components: {
16         newCompo,
17     },
18 }
19 </script>
```

Components

1. Create a component

- In **components** directory, create a new file (e.g., newCompo.vue).
- Add the structure **<template>**, **<script>**, and **<style>**.

2. Export the component, i.e., make it available to be used by other components and views.

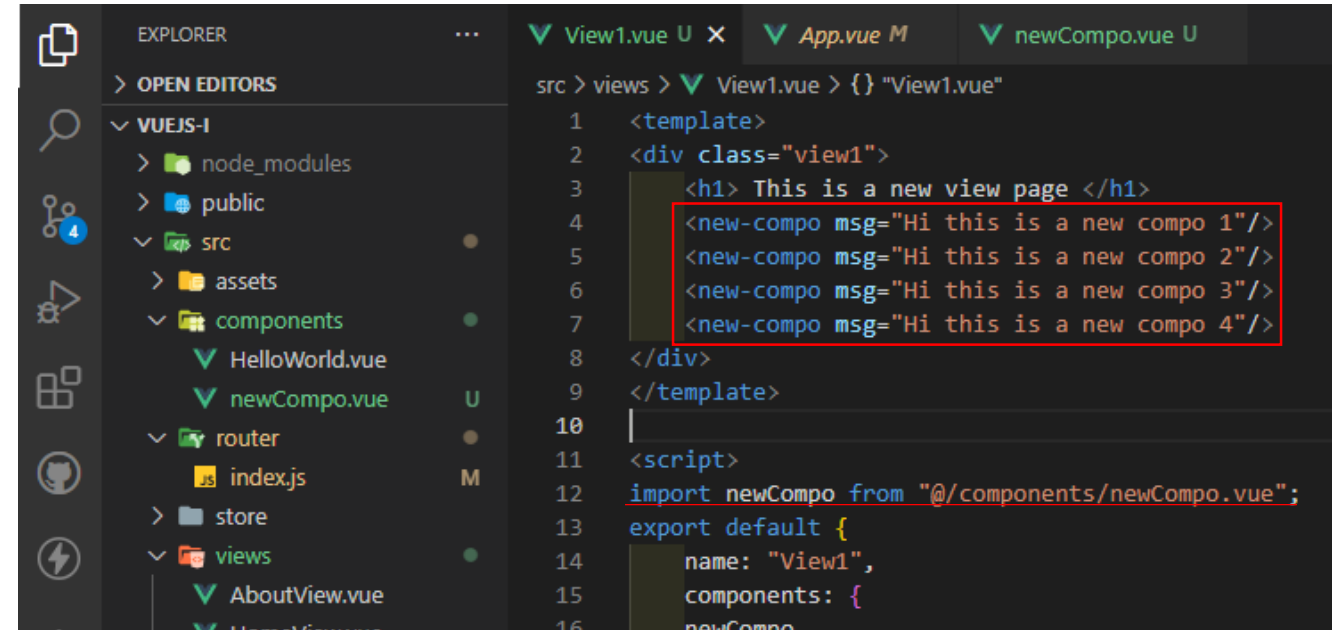
export default {name: "newCompo", }

3. Register/Import the new component in the using component/view.

4. The newly created component can be used in another component or view as a tag using the key it is registered under.

<newCompo />

<new-compo> </new-compo>



The screenshot shows the VS Code interface. The Explorer on the left shows a project structure with a 'components' directory containing 'newCompo.vue'. The main editor shows the content of 'View1.vue', which includes a template with four `<new-compo>` tags and a script section that imports 'newCompo' from '@components/newCompo.vue' and exports it by default with the name 'View1'.

```
src > views > View1.vue > {} "View1.vue"
1  <template>
2  <div class="view1">
3      <h1> This is a new view page </h1>
4      <new-compo msg="Hi this is a new compo 1"/>
5      <new-compo msg="Hi this is a new compo 2"/>
6      <new-compo msg="Hi this is a new compo 3"/>
7      <new-compo msg="Hi this is a new compo 4"/>
8  </div>
9  </template>
10
11  <script>
12  import newCompo from "@components/newCompo.vue";
13  export default {
14      name: "View1",
15      components: {
16          newCompo
```

Home | View1 | About

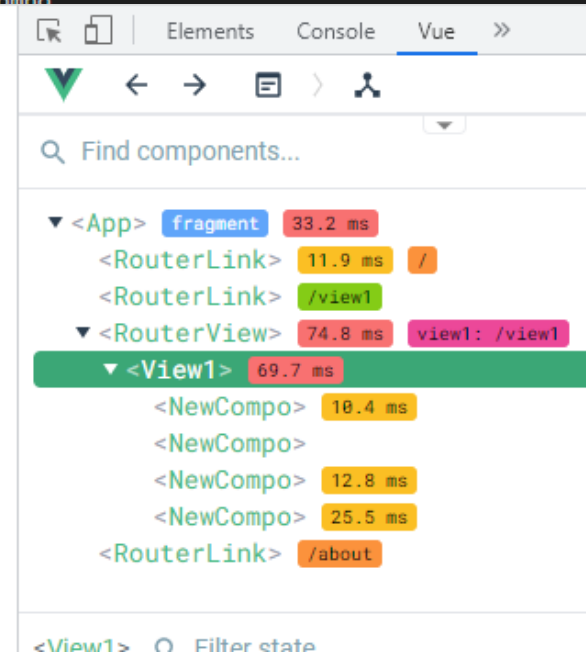
This is a new view page

Hi this is a new compo 1

Hi this is a new compo 2

Hi this is a new compo 3

Hi this is a new compo 4



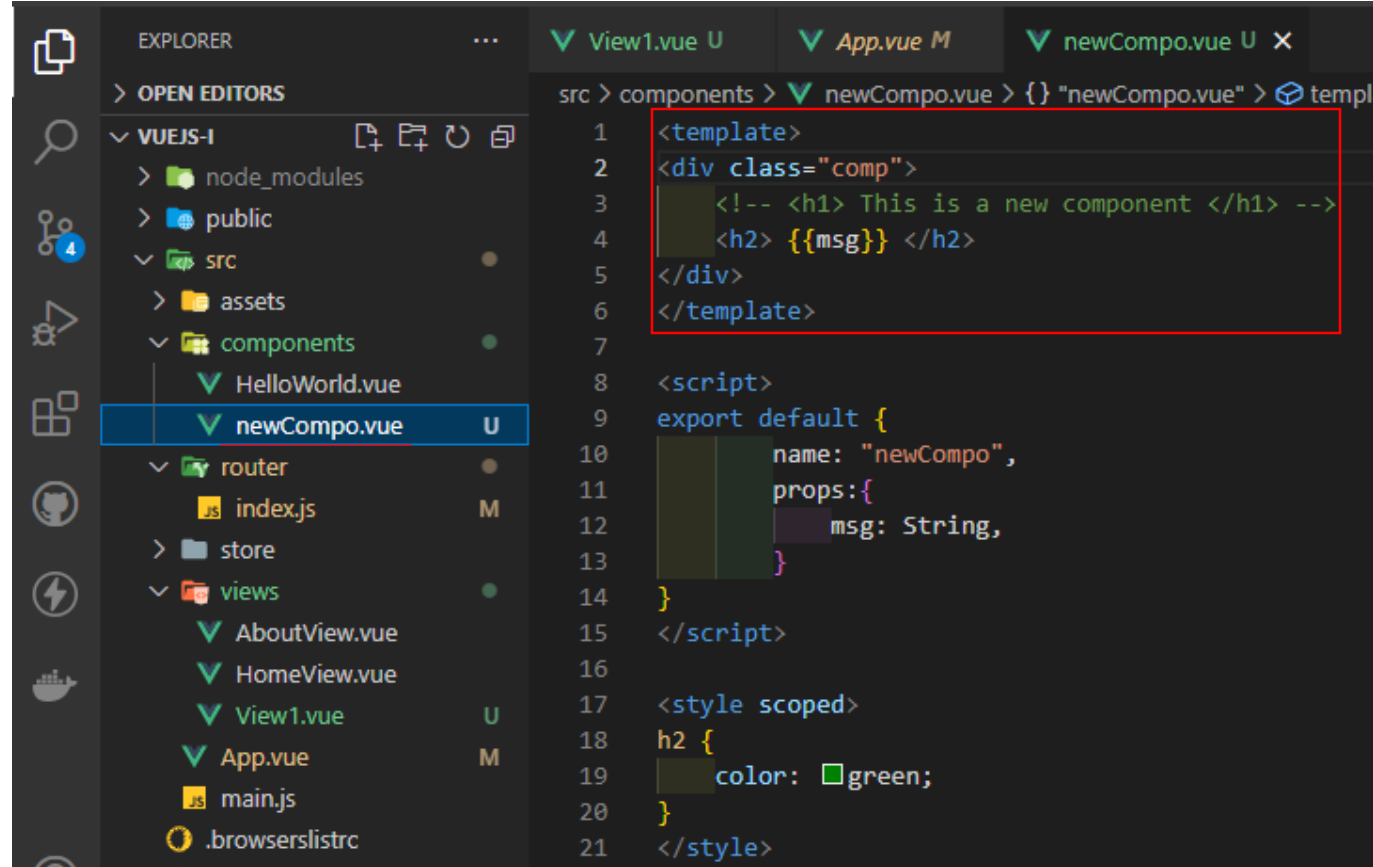
Components - `<template>`

`<template>` is an HTML-based template that allows binding the rendered DOM to the underlying component instance's data.

`<template>` contains valid HTML that can be parsed by browsers and HTML parsers.

To take full advantage of the `<template>`, we need to understand:

- Data declaration and use (Data type checks and other validators).
- Passing data to components (`props: {}`).
- Computed & Watched properties.
- **Directives.**

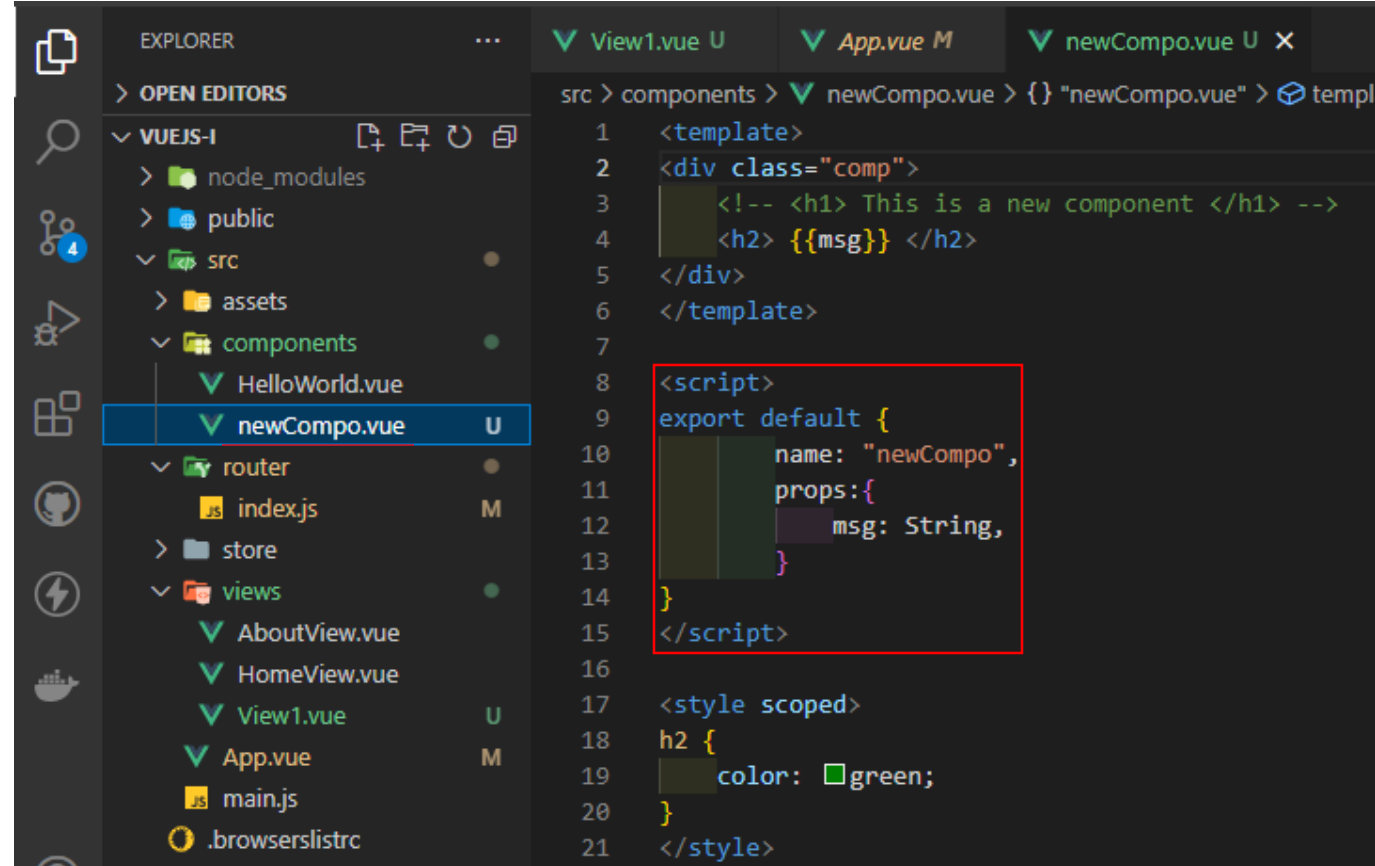


Components - `<script>`

`<script>` each *.vue file may contain a `<script>` block.

`<script>` contains the “**data**” and most of the **logic** () in the component, such as:

- Data declaration (Data type checks and other validators).
- Passing data to components (props: {}).
- Computed & Watched properties
- Methods.
- Lifecycle Hooks.

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project structure with folders like 'node_modules', 'public', 'src', and 'components'. The 'components' folder is expanded, showing 'HelloWorld.vue' and 'newCompo.vue', with 'newCompo.vue' selected. The main editor area shows the code for 'newCompo.vue'. It contains a template section with a div and a script section. The script section is highlighted with a red box and contains the following code:

```
1 <template>
2 <div class="comp">
3   <!-- <h1> This is a new component </h1> -->
4   <h2> {{msg}} </h2>
5 </div>
6 </template>
7
8 <script>
9 export default {
10   name: "newCompo",
11   props: {
12     msg: String,
13   }
14 }
15 </script>
16
17 <style scoped>
18 h2 {
19   color: green;
20 }
21 </style>
```

Components - `<style>`

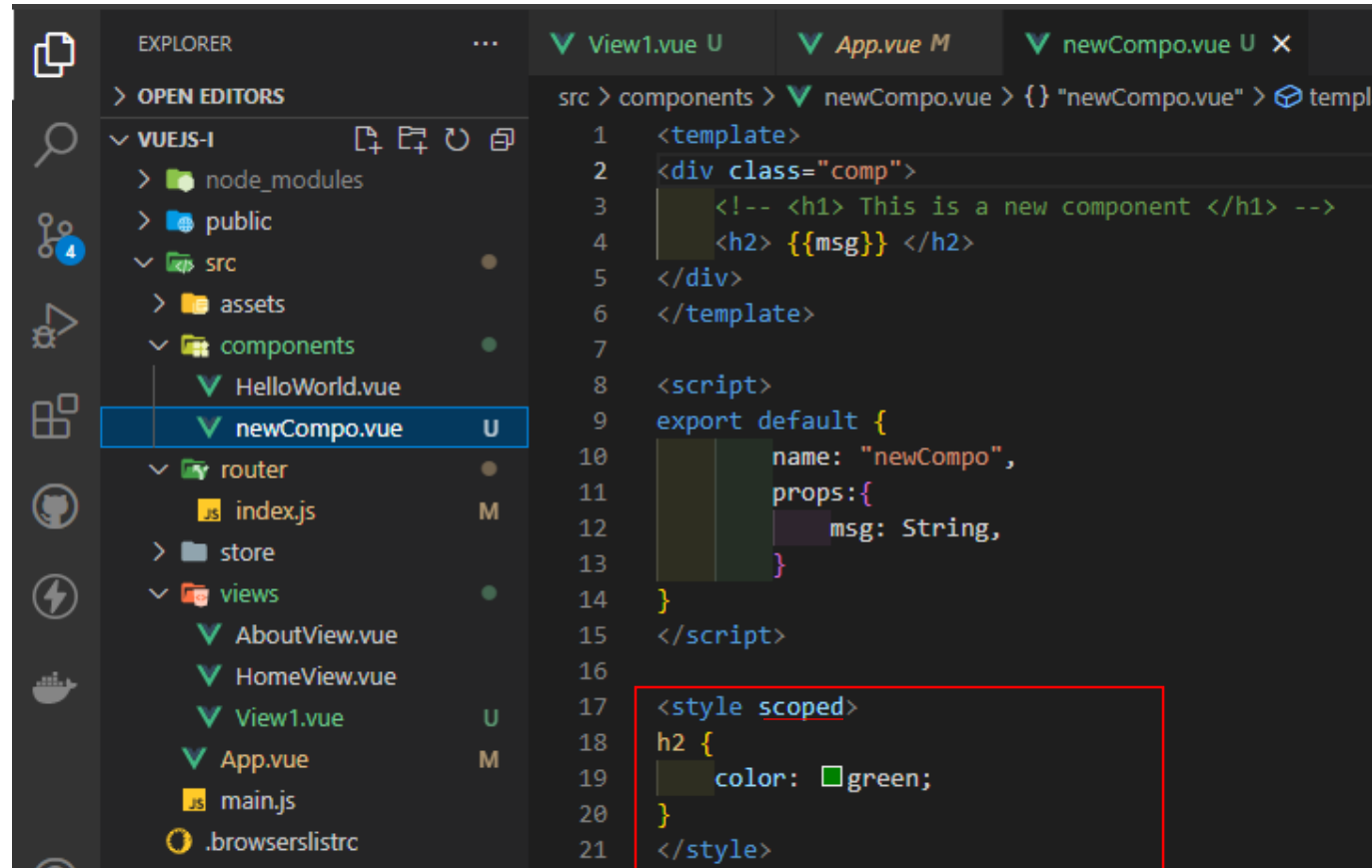
`<style>` each *.vue file may contain a `<style>` block.

`<style>` contains the “styling” of the component. In components, which is written in **CSS**.

CSS is the language that can be used to style an HTML document.

`<style scoped>` is used to limit the application of the defined styling to the component.

keep an eye on the `<style>`

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project structure with a 'components' folder containing 'newCompo.vue', which is selected. The main editor area shows the content of 'newCompo.vue'. The file has three tabs: 'View1.vue', 'App.vue', and 'newCompo.vue'. The code is as follows:

```
1 <template>
2 <div class="comp">
3   <!-- <h1> This is a new component </h1> -->
4   <h2> {{msg}} </h2>
5 </div>
6 </template>
7
8 <script>
9 export default {
10   name: "newCompo",
11   props: {
12     msg: String,
13   }
14 }
15 </script>
16
17 <style scoped>
18   h2 {
19     color: green;
20   }
21 </style>
```

The `<style scoped>` block at the bottom is highlighted with a red rectangle.

Data, Props and Methods

Data declaration and use

Data must be declared as a **function** that returns the data object(s).

Supported data types (String, Number, Boolean, Array, Object, Date)

Why? because if we declared data as an object, it will be shared by reference across all created component instances.

By providing a data function, every time a new instance is created we can call it to return a fresh copy of the initial data.

The most basic form of data binding is text interpolation using the “**Mustache**” `{{}}`, which can be also used for Using JavaScript Expressions.

```
src > components > newCompo.vue > {} "newCompo.vue" > script > default
1  <template>
2  <div class="comp">
3    <!-- <h1> This is a new component </h1> -->
4    <!-- <h2> {{ 'Props: ' + msg }} </h2> -->
5    <h3> {{ 'Data: ' + HiText }} </h3>
6  </div>
7  </template>
8
9  <script>
10 export default {
11   name: "newCompo",
12   props: {
13     msg: String,
14   },
15   data: () => {
16     return {
17       HiText: "We are learning Vue",
18       isVisible: Boolean,
19       objArray: [
20         {id: 1, name: 'name1', type: 'type1', description: 'description1'},
21         {id: 2, name: 'name2', type: 'type2', description: 'description2'},
22         {id: 3, name: 'name3', type: 'type3', description: 'description3'},
23       ],
24     },
25   },
26 </script>
```

Using JavaScript Expressions:
syntax.html#using-javascript-expressions

<https://vuejs.org/guide/essentials/template->

Data declaration and use

Data must be declared as a **function** that returns the data object(s).

Supported data types (String, Number, Boolean, Array, Object, Date)

Why? because if we declared data as an object, it will be shared by reference across all created component instances.

By providing a data function, every time a new instance is created we can call it to return a fresh copy of the initial data.

The most basic form of data binding is text interpolation using the “**Mustache**” `{{}}`, which can be also used for Using JavaScript Expressions.

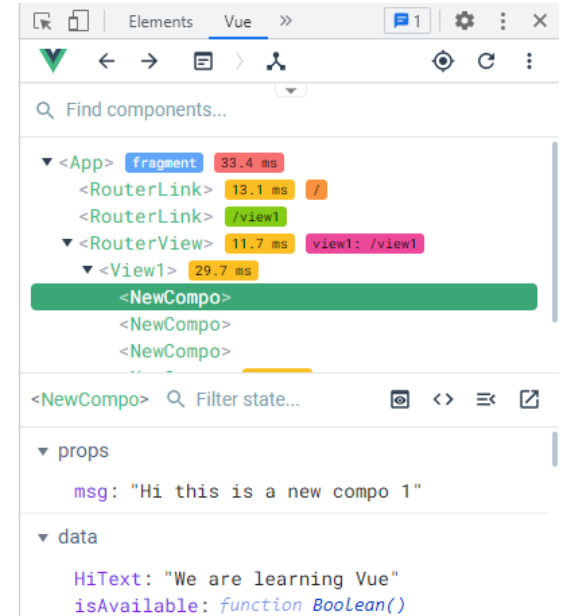
This is a new view page

Data: We are learning Vue

Data: We are learning Vue

Data: We are learning Vue

Data: We are learning Vue



```
15 data:()=>{
16   return{
17     HiText: "We are learning Vue",
18     isAvailable: Boolean,
19     objArray : [
20       {id: 1, name: 'name1', type: 'type1', description: 'description1'},
21       {id: 2, name: 'name2', type: 'type2', description: 'description2'},
22       {id: 3, name: 'name3', type: 'type3', description: 'description3'},
23     ],
24   },
25 }
26 </script>
```

Using JavaScript Expressions:
syntax.html#using-javascript-expressions

<https://vuejs.org/guide/essentials/template->

Props: { }

Props are used to indicate what external data should be passed to the component.

Props vs Data: props are meant to be propagated (passed) and managed from parent components, while data is the component internal state, i.e., the component is responsible for.

Props types: props have types (e.g., String, Number, Boolean, Array, Object, Date, Function).

```
src > components > newCompo.vue > {} "newCompo.vue" > script > default
1 <template>
2   <div class="comp">
3     <!-- <h1> This is a new component </h1> -->
4     <h2> {{ 'Props: ' + msg }} </h2>
5     <h3> {{ 'Data: ' + HiText }} </h3>
6   </div>
7 </template>
8
9 <script>
10 export default {
11   name: "newCompo",
12   props: {
13     msg: String,
14   },
15   data: () => {
16     return {
17       HiText: "We are learning Vue",
18       isVisible: Boolean,
19       objArray: [
20         {id: 1, name: 'name1', type: 'type1', description: 'description1'},
21         {id: 2, name: 'name2', type: 'type2', description: 'description2'},
22         {id: 3, name: 'name3', type: 'type3', description: 'description3'},
23       ],
24     },
25   },
26 </script>
```

Props: { }

Props are used to indicate what external data should be passed to the component.

Props vs Data: props are meant to be propagated (passed) and managed from parent components, while data is the component internal state, i.e., the component is responsible for.

Props types: props have types (e.g., String, Number, Boolean, Array, Object, Date, Function).

```
src > components > newCompo.vue > {} "newCompo.vue" > script > default
1 <template>
2   <div class="comp">
3     <!-- <h1> This is a new component </h1> -->
4     <h2> {{ 'Props: ' + msg }} </h2>
5     <h3> {{ 'Data: ' + HiText }} </h3>
6   </div>
7 </template>
```

This is a new view page

Props: Hi this is a new compo 1

Data: We are learning Vue

Props: Hi this is a new compo 2

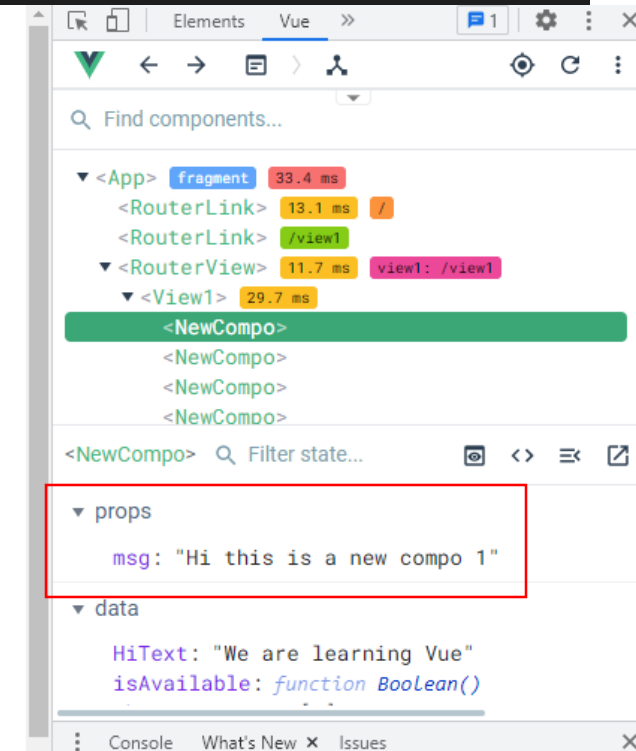
Data: We are learning Vue

Props: Hi this is a new compo 3

Data: We are learning Vue

Props: Hi this is a new compo 4

Data: We are learning Vue



Props checks and other validators

Components can specify requirements for their **props**:

Types checks, Vue checks if a prop has a given type, and will throw a warning if it does not.

Props checks and validators:

Default, If a **default** value is specified, it will be used if the resolved prop value is **undefined**.

Required, All props are optional by default, unless **required: true** is specified.

Custom validator function that takes the prop value as the sole argument.

```
src > components > newCompo.vue > {} "newCompo.vue" > script > default > props > msg >
1  <template>
2  <div class="comp">
3    <!-- <h1> This is a new component </h1> -->
4    <h2> {{ 'Props: ' + msg }} </h2>
5    <h3> {{ 'Data: ' + HiText }} </h3>
6  </div>
7  </template>
8
9  <script>
10 export default {
11   name: "newCompo",
12   props: {
13     msg: {type: String, required: true, default: 'no props were passed',
14        /* validator: (value) => { /^[a-zA-Z]+$/.test(value)}, */
15        /* validator: (value) => {return value == ''}, */
16     validator: (value) => {return value != ''},
17   },
18 }
```

Note: when prop validation fails, Vue will produce a console warning.

Methods

Methods can be accessed directly in the Vue instance, or they can be used in directive expressions.

All methods will have their this context automatically bound to the Vue instance.

```
src > components > newCompo.vue > {} "newCompo.vue" > script > default >
1 <template>
2 <div class="comp">
3   <!-- <h1> This is a new component </h1> -->
4   <h2> {{ 'Props: ' + msg }} </h2>
5   <h3> {{ 'Data: ' + HiText }} </h3>
6   <button id="example" v-on:click="sayHi">Say Hi</button>
7 </div>
8 </template>
9
10 <script>
11 export default {
12   name: "newCompo",
13   props: { ...
18   },
19   data: () => {
20     return {
21       HiText: "We are learning Vue",
22       isAvailable: Boolean,
23       objArray : [ ...
27     ],
28   },
29   methods: {
30     sayHi: function (event) {
31       // `this` inside methods point to the Vue instance
32       alert('Hello ' + this.HiText + '!')
33     }
34   },
35 }
```

Directives

A directive is a special token in the markup that allows attaching a specified behaviour to that DOM elements

<https://vuejs.org/api/built-in-directives.html>

Directives: v-text & v-html

v-text: updates the element's text content, so it will overwrite any existing content inside the element.

```
<span v-text="msg"></span>
```

v-text vs. {{}}, if you need to update a part of text content, it is better to **{{}}**.

v-html: updates the element's innerHTML, and it expects String.

```
<p v-html="messageHtml"></p>
```

```
<!-- messageHtml: "<h1> Hi I am an HTML<h1/>" -->
```

```
<!-- <v-text> -->
  <p v-text= "message"> </p>

<!-- <v-html> -->
  <p v-html= 'messageHtml'> </p>
```

```
data: function() {
  return {
    message: "Hi, I am a v-text",
    messageHtml: "<h1> Hi, I am a v-HTML<h1/>",
  }
}
```

Hi, I am a v-text

Hi, I am a v-HTML

Directives: v-on

v-on (Shorthand: “@”) Attaches an event listener to an element. The event type is denoted by the argument. The expression can be a method or an inline statement.

```
<!-- method handler -->  
<button v-on:click="dosomething"></button>
```

```
<!-- inline statement -->  
<button v-on:click=" dosomething('hello', $event)"></button>
```

```
<!-- v-on -->  
<p @click= 'count++'> Hello {{nameClick}}, you have clicked {{count}} times </p>
```

Hello John, you have clicked 7 times

```
data: function() {  
  return {  
    nameClick: 'John',  
    count: 0,  
  }  
}
```

Directives: v-show & v-once

v-show: toggle the element's visibility based on the truthfulness of the expression value.

v-show works by setting the display CSS property via inline styles, and will try to respect the initial display value when the element is visible. It also triggers transitions when its condition changes.

```
<h1 v-show="true"> Hello! </h1>
```

```
<h1 v-show ="count>4"> count: {{count}}</h1>
```

v-once: render the element and component once only, and skip future updates.

On subsequent re-renders, the element/component and all its children will be treated as static content and skipped.

```
<new-compo v-once msg = "This is a new compo" />
```

```
<!-- <v-show> -->
<h1 v-show="true">Hello!</h1>
<h1 v-show ="count>4"> v-show test count: {{count}}</h1>
```

v-show test count: 7

```
<!-- v-once -->
<h2> v-once through componenet </h2>
<new-compo v-once msg="Hi this is a new compo - v-once"/>
```

v-once through componenet

Props: Hi this is a new compo - v-once

Data: We are learning Vue

Say Hi

Directives: **v-if**, **v-else-if** & **v-else**

v-if: conditionally render an element or a template fragment based on the truthy-ness of the expression value.

`<p v-if="Math.random() > 0.4">` Appear if random number bigger than 0.4 `</p>`

v-else-if - Restriction: previous sibling element must have **v-if** or **v-else-if**.

`<p v-else-if="Math.random() == 0.4">` Appear if random number equal to 0.4 `</p>`

v-else: denote the "else block" for **v-if** or a **v-if** / **v-else-if** chain.

`<p v-else>` "random number not bigger than 0.4" `</p>`

```
<p v-if= "Math.random() > 0.4"> Appear if random number bigger than 0.4  </p>
<p v-else-if = "Math.random() == 0.4"> Appear if random number equal  to 0.4  </p>
<p v-else> "random number not bigger than 0.4" </p>
```

Directives: v-bind & v-model

v-bind (Shorthand **:**): dynamically bind one or more attributes, or a component prop to an expression.

`<!-- bind an attribute -->`

``

`<!-- shorthand -->`

`<input type = "text" :value="writeSomething">`

v-model: create a two-way binding on a form input element or a component.

`<input type = "text" v-model ="writeSomething">`



Hard coded Image source

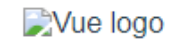


Image source provided without bind



Image source provided with bind

using v-bind

using v-model

You wrote

Directives: v-for

v-for: render the element or template block multiple times based on the source data.

View: **view1.vue**

```
<como-list= :compoArray="compoArray"/>
...
data: function() {
  return {
    compoArray : [
      {id: 1, msg: 'msg description 1'}, ...]
  }
}
```

Component: **compoArray.vue**

```
<ul>
<li v-for= "compo in compoArray" :key="compo.id">
<span> <b>Id:</b>{{compo.id}} </span><br/>
<span> <b> Msg desc.:</b> {{compo.msg}} </span>
</li>
</ul>
```

```
<!-- v-for -->
<h2> v-for, props: and components </h2>
<como-list :compoArray="compoArray"/>
```

```
compoArray : [
  {id: 1, msg: 'msg description 1'},
  {id: 2, msg: 'msg description 2'},
  {id: 3, msg: 'msg description 3'},
],
}
```

```
src > components > ✓ compoList.vue > {} "compoList.vue" > template > div#comp-list >
1 <template>
2 <div id= "comp-list">
3 <h1>This is a list of Componenet </h1>
4 <ul>
5 <li class="item" v-for= "compo in compoArray" :key="compo.id">
6 <span class="id"> <b>Id:</b> {{compo.id}} </span><br/>
7 <span class="msg"> <b>Msg description:</b> {{compo.msg}} </span>
8 </li>
9 </ul>
10 </div>
11 </template>
12
13 <script>
14 export default {
15   name: "compolist",
16   props: ["compoArray"],
17   data:()=>{
18     return{
19     }},
20   };
21 </script>
```


Directives: v-for

v-for: render the element or template block multiple times based on the source data.

View: **view1.vue**

```
<como-list= :compoArray="compoArray"/>
...
data: function() {
  return {
    compoArray: [
      {id: 1, msg: 'msg description 1'}, ...]
    }
  }
}
```

Component: **compoArray.vue**

```
<ul>
<li v-for= "compo in compoArray" :key="compo.id">
<span> <b>Id:</b>{{compo.id}} </span><br/>
<span> <b>Msg desc.:</b> {{compo.msg}} </span>
</li>
</ul>
```

```
<!-- v-for -->
<h2> v-for, props: and components </h2>
<compo-list :compoArray="compoArray"/>
```

```
compoArray : [
  {id: 1, msg: 'msg description 1'},
  {id: 2, msg: 'msg description 2'},
  {id: 3, msg: 'msg description 3'},
]
```

```
src > components > ✓ compoList.vue > {} "compoList.vue" > template > div#comp-list >
1 <template>
2 <div id= "comp-list">
3 <h1>This is a list of Componenet </h1>
4 <ul>
5 <li class="item" v-for= "compo in compoArray" :key="compo.id">
6 <span class="id"> <b>Id:</b> {{compo.id}} </span><br/>
7 <span class="msg"> <b>Msg desc.:</b> {{compo.msg}} </span>
8 </li>
9 </ul>
10 </div>
11 </template>
12
13 <script>
14 export default {
15   name: "compoList",
16   props: ["compoArray"],
17   data: () => {
18     return {
19     },
20   },
21 </script>
```

v-for, props: and components

This is a list of Componenet

Id: 1
Msg description: msg description 1
Id: 2
Msg description: msg description 2
Id: 3
Msg description: msg description 3

Computed and watched properties

<https://vuejs.org/guide/essentials/computed.html>

<https://vuejs.org/guide/essentials/watchers.html>

Computed properties

computed properties enable you to create a property that can be used to modify, manipulate, and display data within your components in a readable and efficient manner.

computed properties is a way to reduce the complexity of the `<template>` as it can become bloated and hard to maintain.

Unlike **methods**, **computed properties** will be “re-calculated” anytime some data changes in the component.

Computed properties

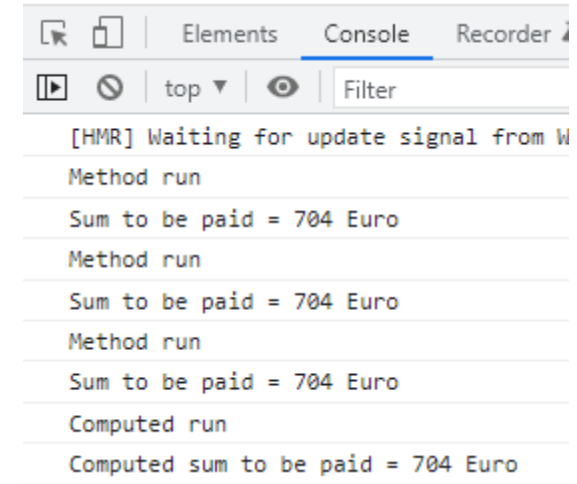
computed properties enable you to create a property that can be used to modify, manipulate, and display data within your components in a readable and efficient manner.

computed properties is a way to reduce the complexity of the `<template>` as it can become bloated and hard to maintain.

Unlike **methods**, **computed properties** will be “re-calculated” anytime some data changes in the component.

```
<p> {{toBePaid()}} </p>
<p> {{toBePaid()}} </p>
<p> {{toBePaid()}} </p>

<p> {{toBePaidComputed}} </p>
<p> {{toBePaidComputed}} </p>
<p> {{toBePaidComputed}} </p>
```

A screenshot of a web browser's developer console. The 'Console' tab is active, showing a sequence of log messages. The messages alternate between 'Method run' and 'Sum to be paid = 704 Euro'. The final message is 'Computed sum to be paid = 704 Euro', indicating that the computed property was triggered after several method calls.

[HMR] Waiting for update signal from W

Method run

Sum to be paid = 704 Euro

Method run

Sum to be paid = 704 Euro

Method run

Sum to be paid = 704 Euro

Computed run

Computed sum to be paid = 704 Euro

```
methods: {
  toBePaid: function () {
    console.log("Method run");
    var Methodmoney = "Sum to be paid = " + (this.billsTobepaid[0].price + this.billsTobepaid[1].price) + " Euro";
    console.log(Methodmoney)
    return Methodmoney;
  }
},

computed:{
  toBePaidComputed: function () {
    console.log("Computed run");
    var money = "Computed sum to be paid = " + (this.billsTobepaid[0].price + this.billsTobepaid[1].price) + " Euro";
    console.log(money)
    return money;
  }
}
```

Watched properties (Watchers)

Watchers allow the developers to listen to the component data and run whenever they change a particular property.

Watcher is a unique Vue.js feature that lets you keep an eye on one property of the component state and run a function when that property value changes

```
<!-- Watched properties (Watchers) -->
<h2> Watched properties (Watchers) </h2>

<label> Pay in Euro </label>
<input type="text" v-model="Euro"><br>

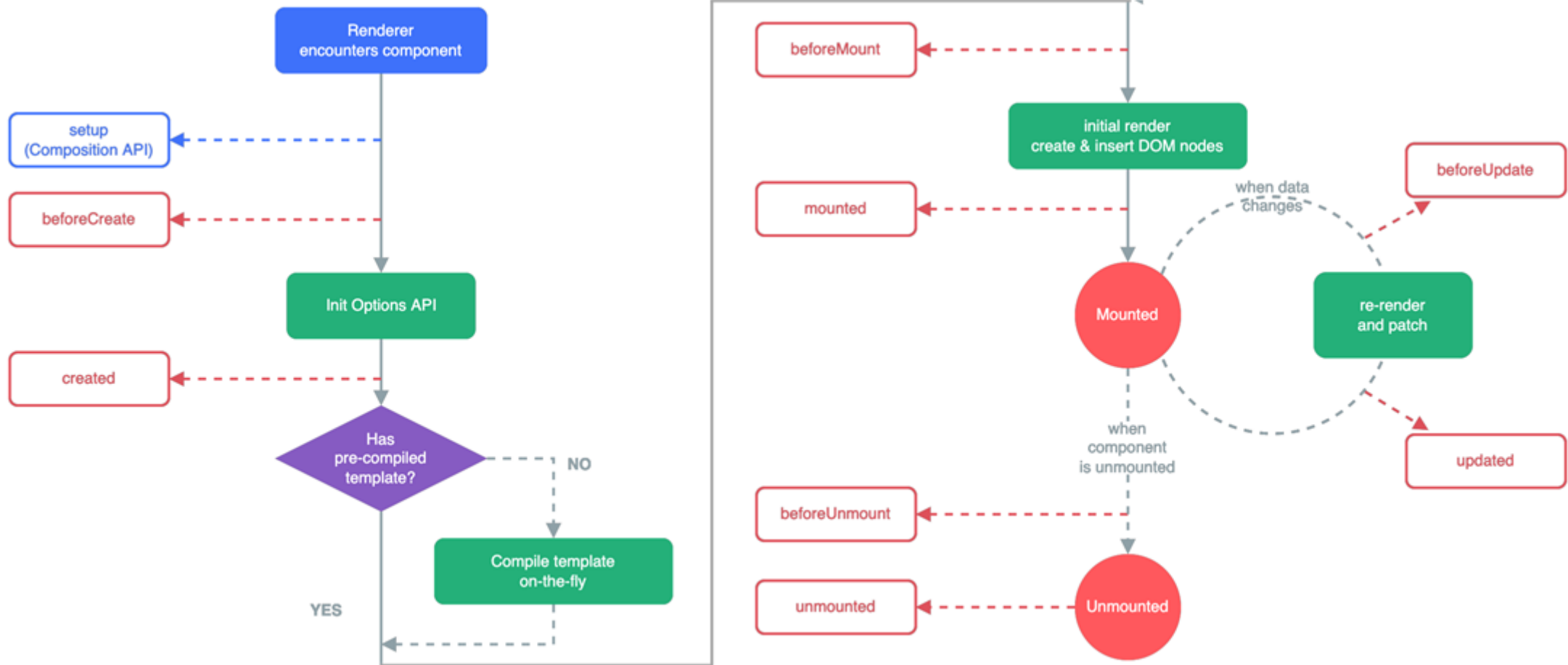
<label> Pay in US Dollar </label>
<input type="text" v-model="Dollar"><br>
```

```
watch:{
  Euro: function(w){
    this.Dollar = w *2;
  },
  Dollar: function(w){
    this.Euro = w /2;
  },
}
```

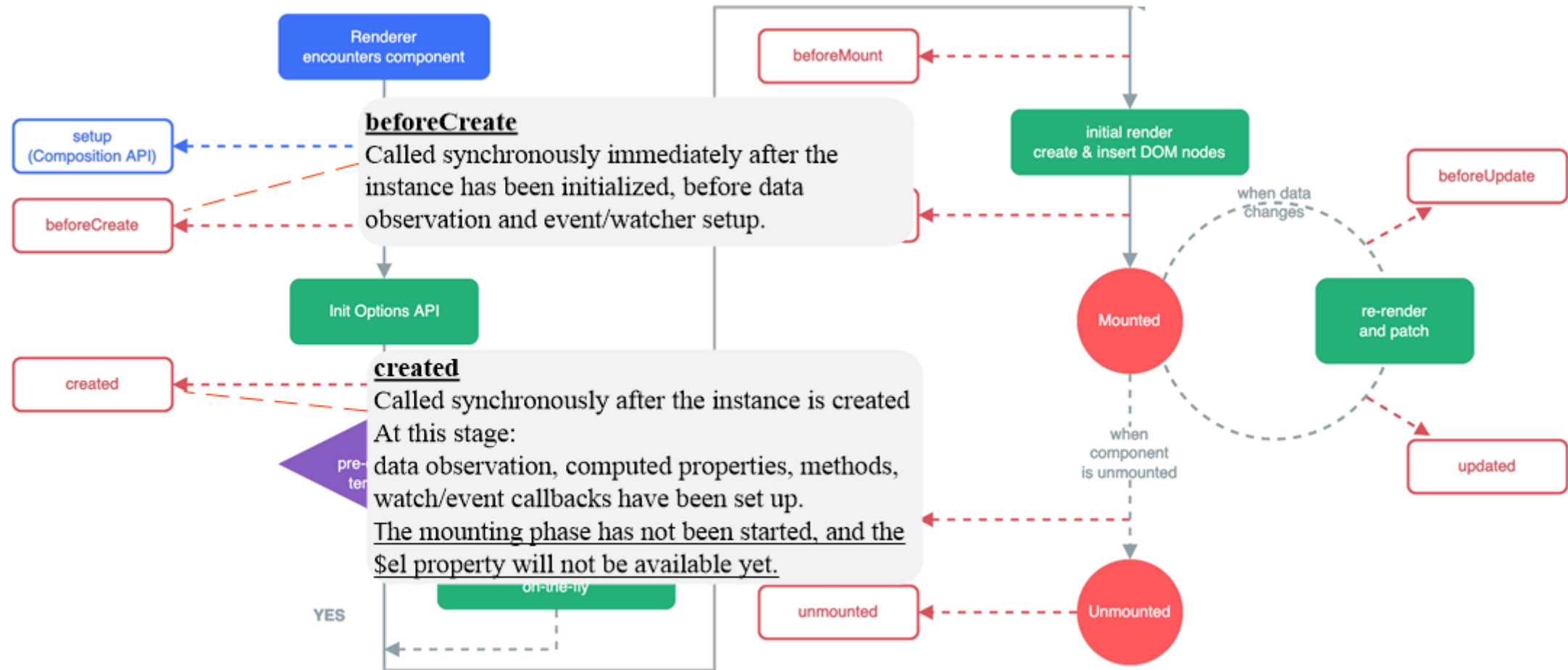
Lifecycle hooks

<https://vuejs.org/api/options-lifecycle.html>

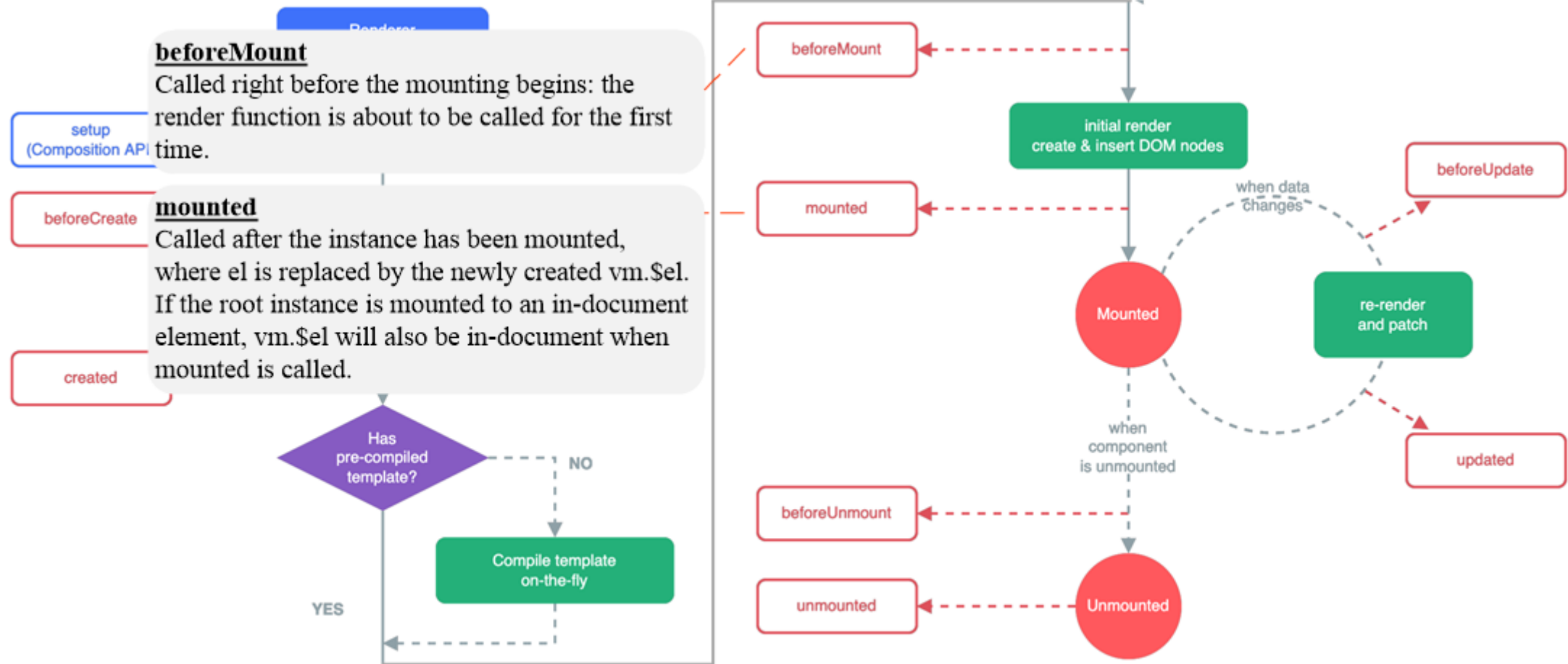
Lifecycle hooks



Lifecycle hooks

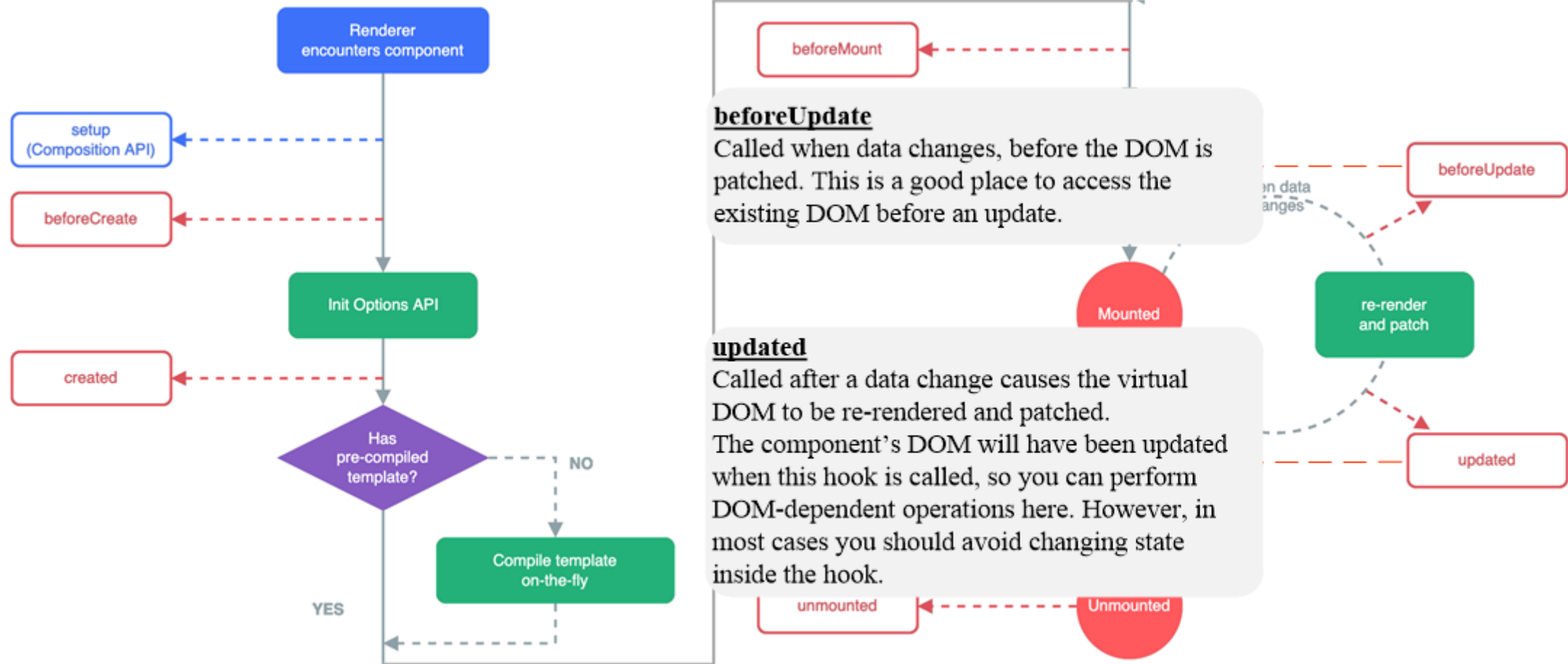


Lifecycle hooks

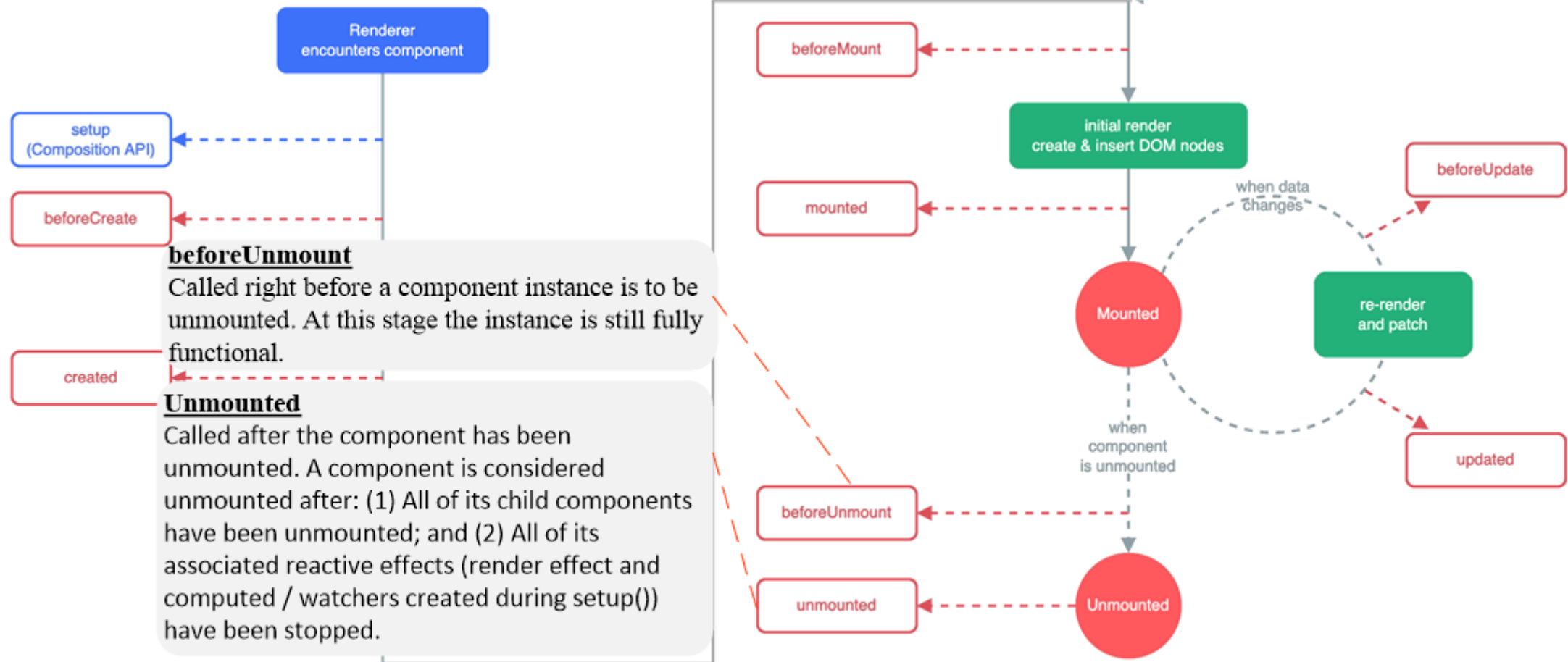


mounted() is commonly used for fetching data from “external sources” to be used in mounted elements.

Lifecycle hooks



Lifecycle hooks



NPM

Node Package Manager

npm¹ is a package manager for the JavaScript programming language.

npm has its own command-line interface (CLI).

npm offers a registry that is an online database of public and paid-for private package.

Why **npm** is very important: it allows adapting packages of code for your apps, or incorporating packages as they are.

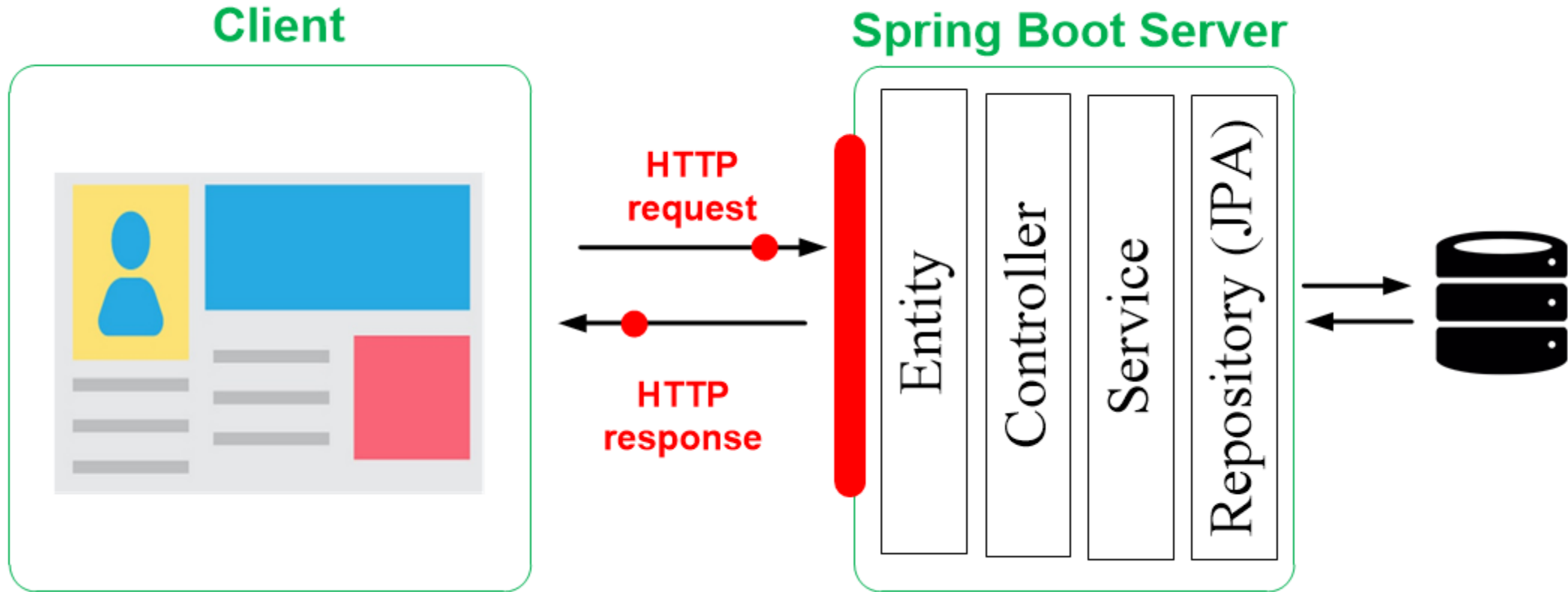


¹<https://www.npmjs.com/>

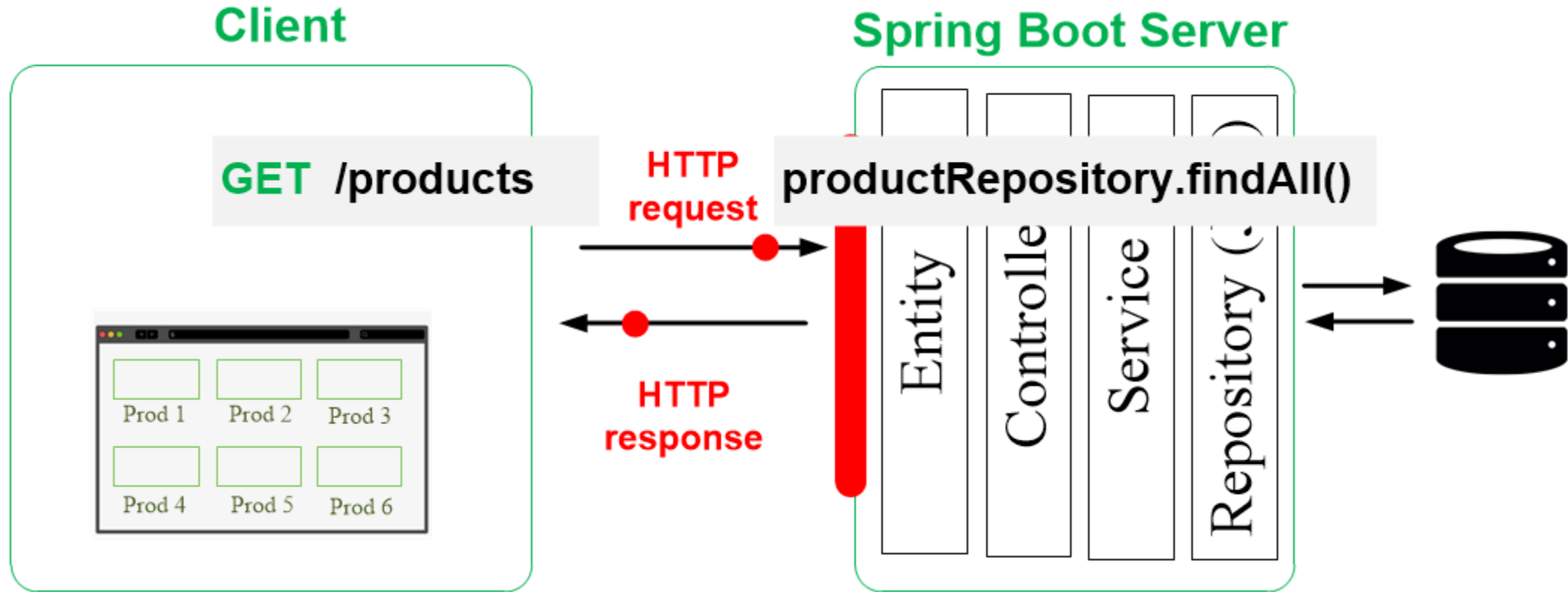
Vue.js

A practical example

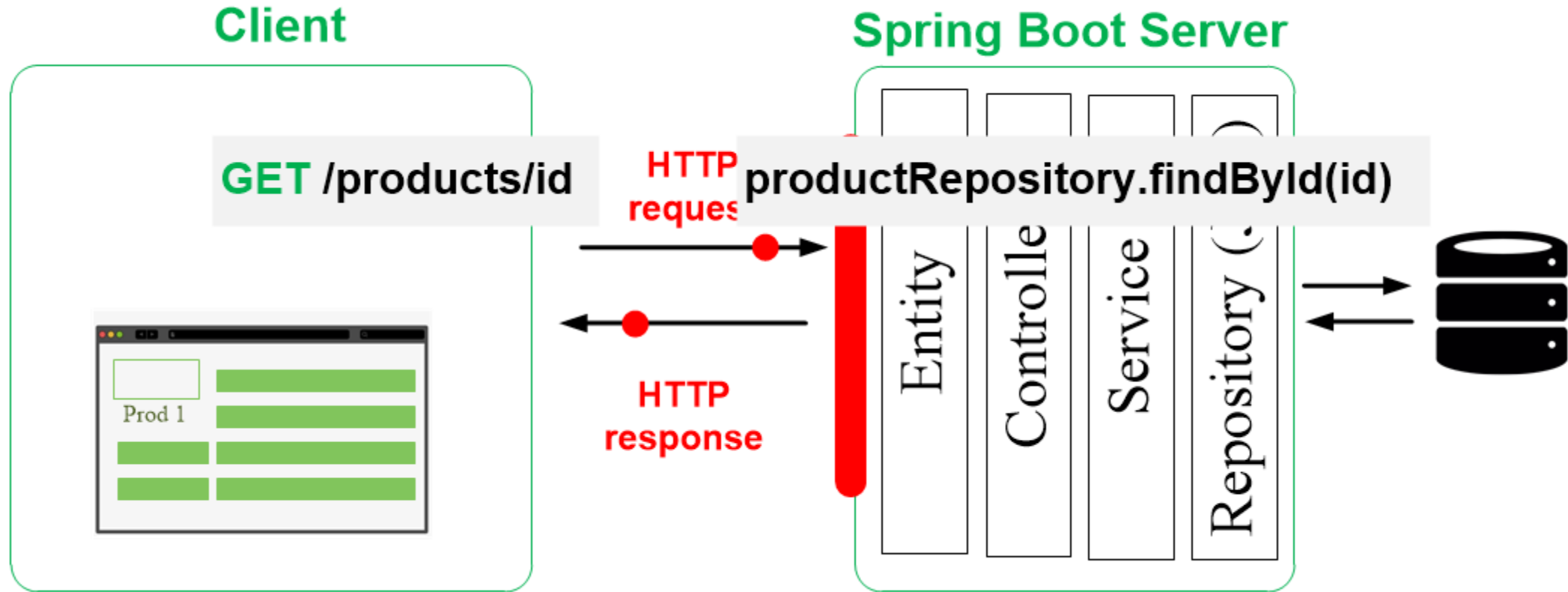
The Client side



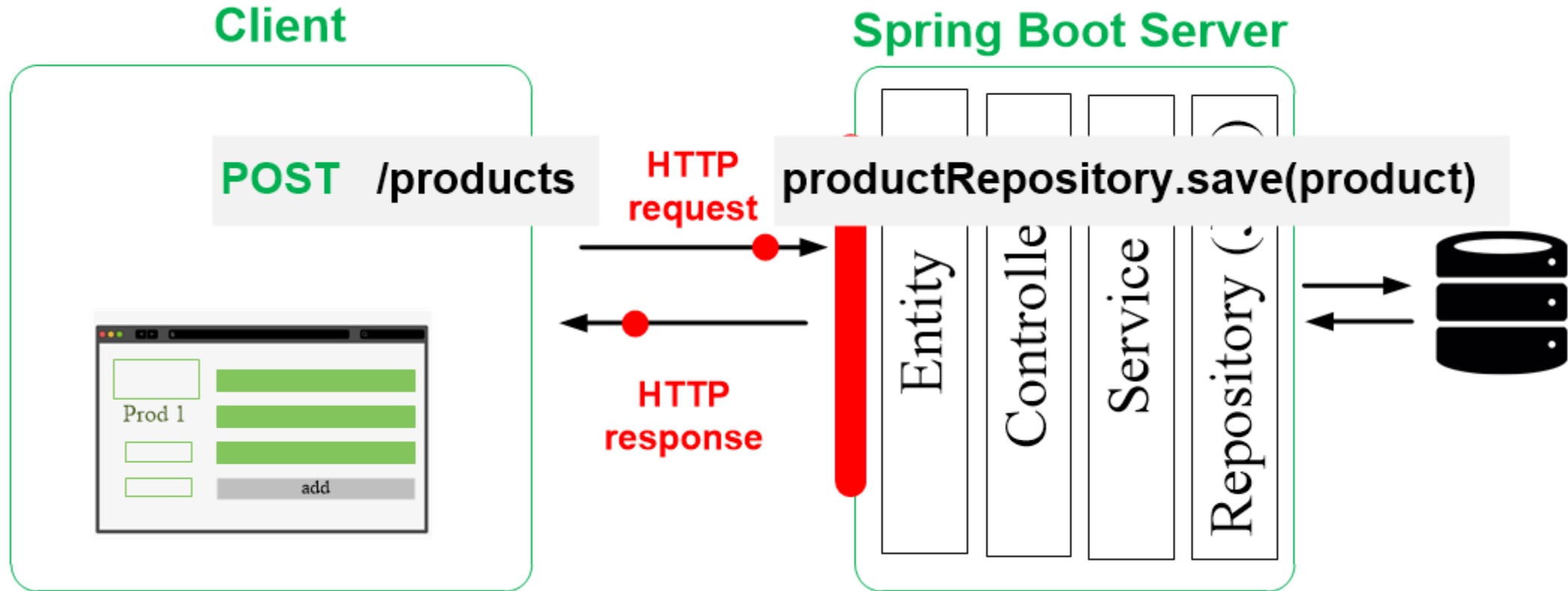
The Client side



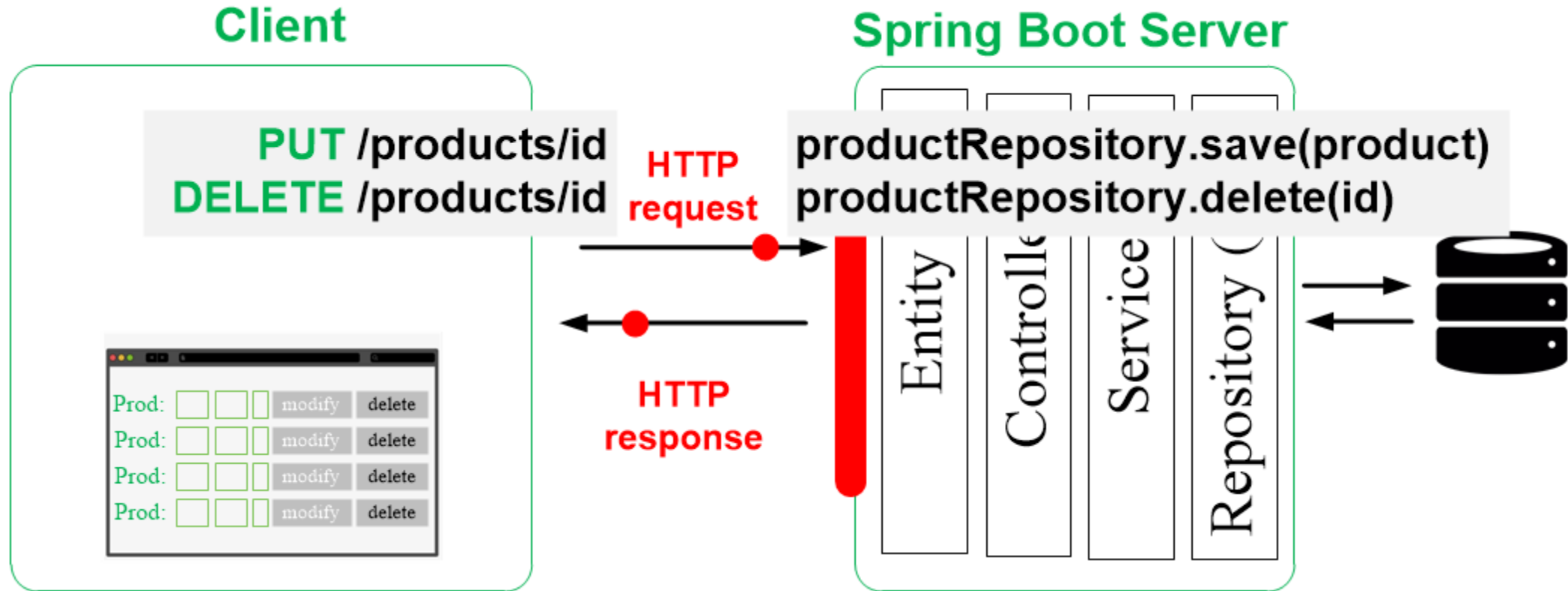
The Client side



The Client side

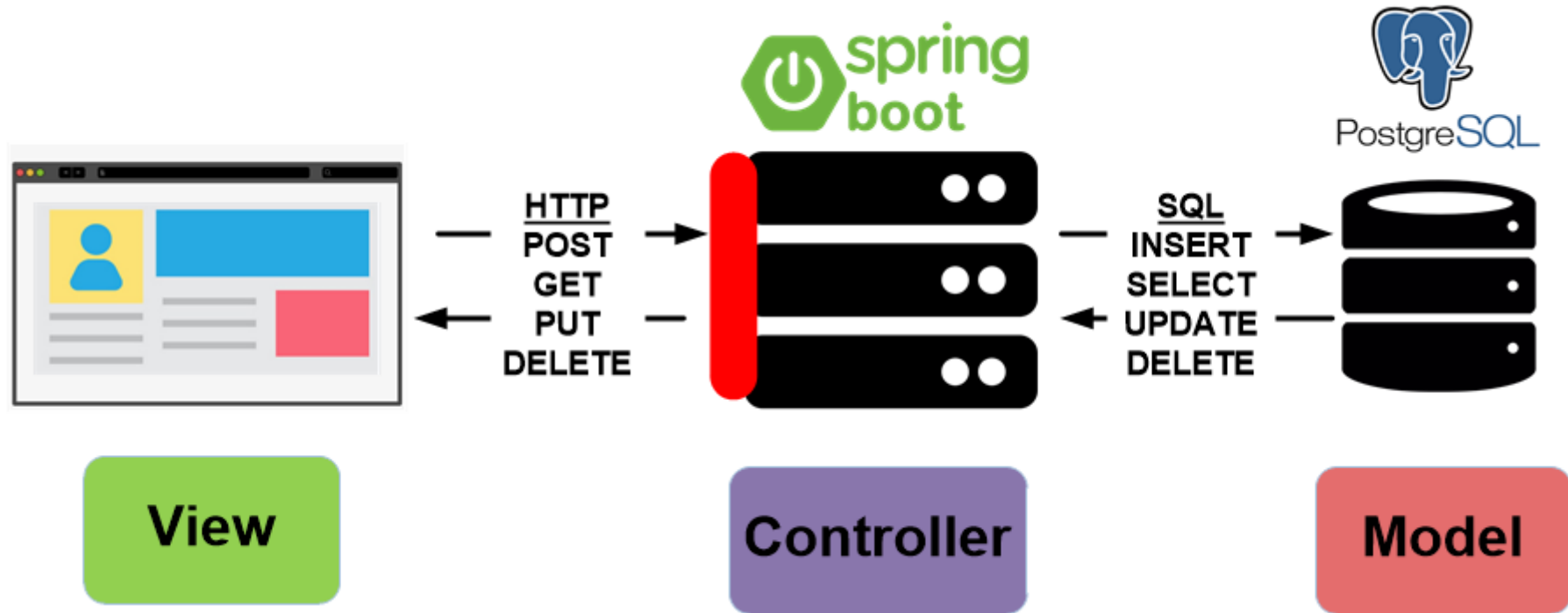


The Client side

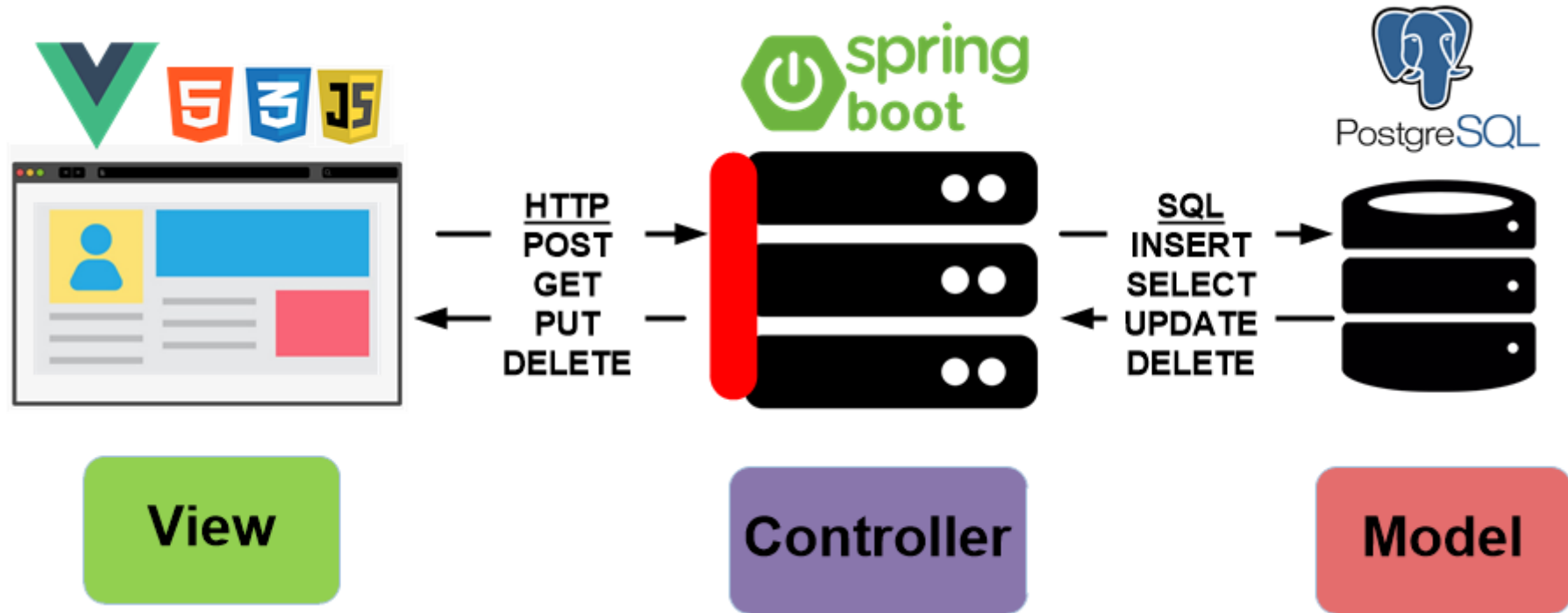


Building an MVC application

Building an MVC application



Building an MVC application



Building an MVC application

We will develop a simple App that includes:

The View - a front-end (Vue.js App) allows to:

- **Create** a product (content) in the database;
- **Fetch** and present all products from the database;
- **Fetch** and present a specific product from the database;
- **Update** a specific product in the database; and
- **Deletes** a specific product from the database.

The Controller - a backend (SpringBoot App) allows for handling the previous requests.

The Model - a database (Postgres) that contain the data of our App.

Building an MVC application

We will develop a simple App that includes:

The View - a front-end (Vue.js App) allows to:

- **Create** a product (content) in the database;
- **Fetch** and present all products from the database;
- **Fetch** and present a specific product from the database;
- **Update** a specific product in the database; and
- **Deletes** a specific product from the database.

The Controller - a backend (SpringBoot App) allows for handling the previous requests.

The Model - a database (Postgres) that contain the data of our App.

Building an MVC application- the Controller

Method	URI/PATH	Actions
POST	/api/products	Creates a new product
GET	/api/products	Fetches all products
GET	/api/products/:id	Fetches a product based on its id
PUT	/api/products/:id	Updates a product based on its id
DELETE	/api/products/:id	Deletes a product based on its id

Building an MVC application- the Controller

Solve the CORS issue

If you are running both the front-end and back-end applications on the same computer, they will be using different ports. This cause a CORS (Cross-Origin Resource Sharing) error.

To solve this issue, you need to use **@CrossOrigin** tag to configure allowed origins.

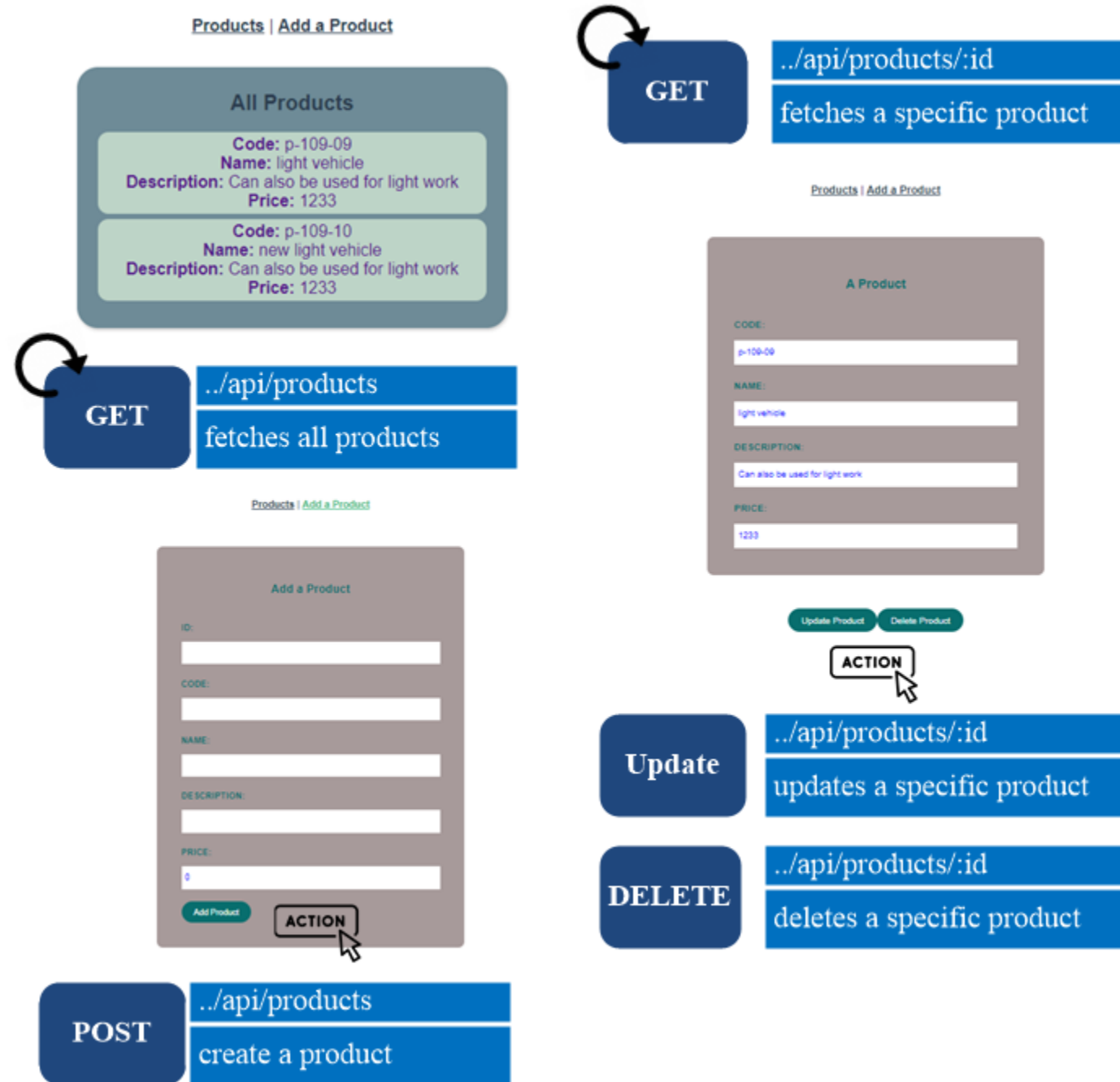
It should be used before each @RestController

```
...  
//@CrossOrigin(origins =  
                "http://localhost:8082")  
@CrossOrigin(origins = "*")  
@RestController  
...
```

The View

Three **views** are required:

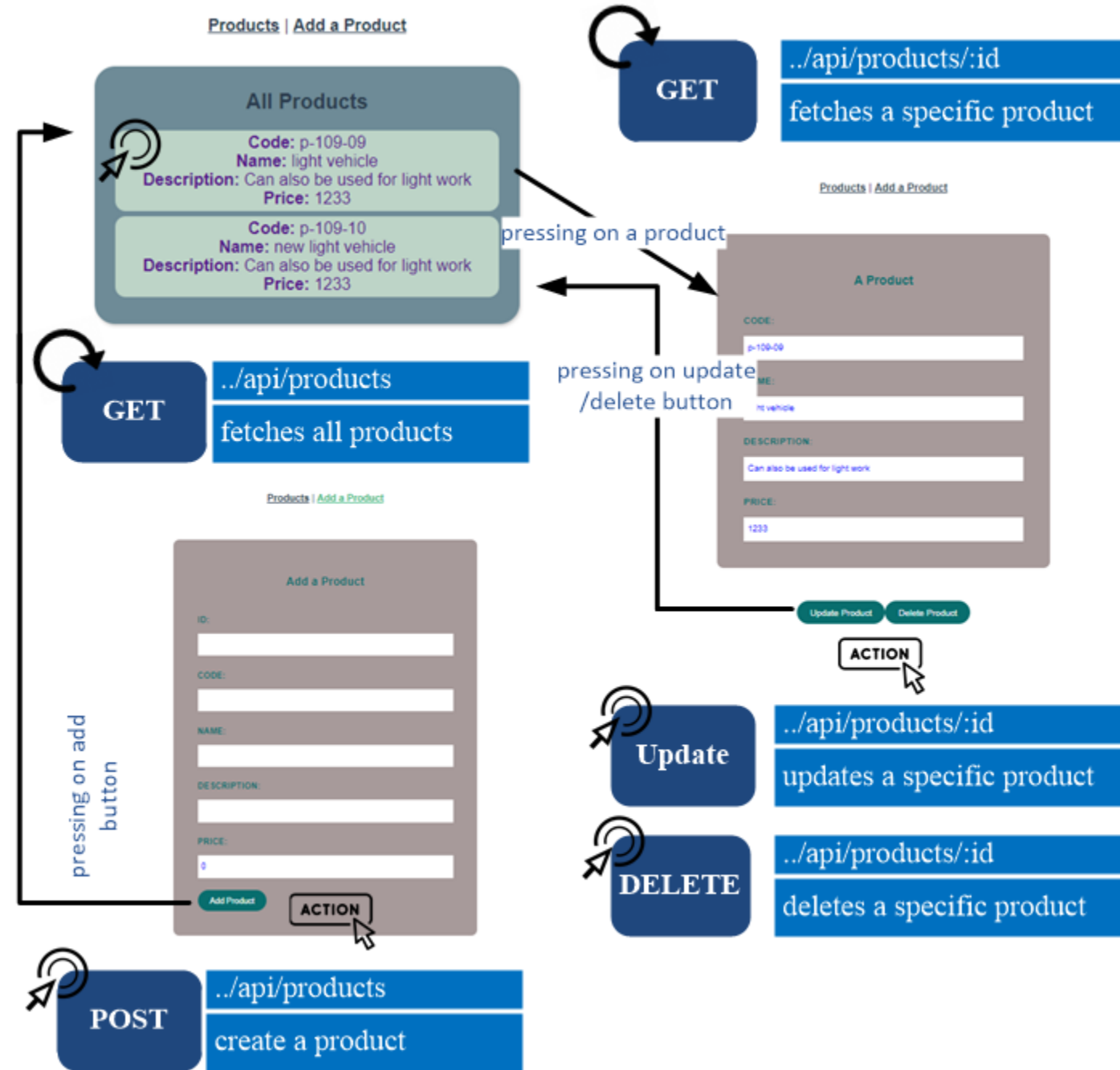
1. **All products:** fetches and presents all products. Pressing on any product, will direct us to the **single product view** with the **id** of the product as a route parameter.
2. **A single product:** fetches and presents a single product based on the passed **id**. This view also contains **update** and **delete** buttons, when pressed on, the product will be updated/deleted and we will be directed to the **All products view**.
3. **A create product:** allows creating a new product, and contains an **add product** button, when pressed on, the product will be added and we will be directed to the **All posts view**.



The View

Three **views** are required:

1. **All products**: fetches and presents all products. Pressing on any product, will direct us to the **single product view with the id of the product** as a route parameter.
2. **A single product**: fetches and presents a single product based on the passed id. This view also contains **update** and **delete** buttons, when pressed on, the product will be updated/deleted and we will be directed to the **All products view**.
3. **A create product**: allows creating a new product, and contains an **add product** button, when pressed on, the product will be added and we will be directed to the **All posts view**.



The View - AllProducts.vue

```
<template>
  <div class="AllProducts">
    <div id="Products-list">
      <h1>All Products</h1>
      <ul>
        <div class="item" v-for="product in products" :key="product.id">
          <a class="singleproduct" :href="'/api/aproduct/' + product.id">
            <span class="code"> <b>Code:</b> {{product.code }} </span> <br/>
            <span class="name"> <b>Name:</b> {{product.name }} </span> <br/>
            ...
          </a>
        </div>
      </ul>
    </div>
  </div>
</template>
```

1. All products: fetches and presents all products. Pressing on any product, will direct us to the **single product view with the id of the product** as a route parameter.

The View - AllProducts.vue

```
<script>
  export default {
    name: "AllProducts",
    data() { return {products:[],}; },
    methods: {
      fetchProducts() {
        fetch(`http://localhost:8082/api/products/`)
          .then((response) => response.json())
          .then((data) => (this.products = data))
          .catch((err) => console.log(err.message));
      },
    },
    mounted() {
      this.fetchProducts();
      console.log("mounted");
    },
  };
</script>
```

1. All products: fetches and presents all products. Pressing on any product, will direct us to the **single product view with the id of the product** as a route parameter.

The View - AProduct.vue

```
<template>
  <div class="A Product">
    <div id="form">
      <h3>A Product </h3>
      <label for="code"> Code : </label>
      <input name="code" type="text" id=" code " required v-model="product.code" />
      <label for="name"> Name : </label>
      <input name="name" type="text" id="name" required v-model="product.name" />
      ...
    </div>
    <div>
      <button @click="updateProduct" class="updateProduct">Update Product</button>
      <button @click="deleteProduct" class="deleteProduct">Delete Product</button>
    </div>
  </div>
</template>
```

2. A single product: fetches and presents a single product based on the passed id. This view also contains **update** and **delete buttons**, when pressed on, the product will be updated/deleted and we will be directed to the **All products view**.

The View - AProduct.vue

```
<script>
export default {
  name: "AProduct",
  data() {return {product: {code: "", name: "", description: "", price: }, }; },
  methods: {
    fetchAProduct(id) {
      fetch(`http://localhost:8082/api/products/${id}`)
        .then((response) => response.json())
        .then((data) => (this.product = data))
        .catch((err) => console.log(err.message));},
  },
}
```

2. A single product: fetches and presents a single product based on the passed id. This view also contains **update** and **delete buttons**, when pressed on, the product will be updated/deleted and we will be directed to the **All products view**.

The View - AProduct.vue

```
updateProduct() {
  fetch(`http://localhost:8082/api/products/${this.product.id}`, {
    method: "PUT", headers: { "Content-Type": "application/json", },
    body: JSON.stringify(this.product), })
    .then((response) => {console.log(response.data);
      this.$router.push("/api/allproducts"); })
    .catch((e) => {console.log(e); });},

deleteProduct() {
  fetch(`http://localhost:8082/api/products/${this.product.id}`, {
    ... // to be done in the lab
  },

mounted() {
  this.fetchAProduct(this.$route.params.id);
},
};
</script>
```

2. A single product: fetches and presents a single product based on the passed id. This view also contains **update** and **delete buttons**, when pressed on, the product will be updated/deleted and we will be directed to the **All products view**.

The View - AddProduct.vue

```
<template>
  <div class="form">
    <h3>Add a Post</h3>
    <label for="code">Code: </label>
    <input name="code" type="text" id="code" required v-model="product.code" />
    <label for="name">Name: </label>
    <input name="name" type="text" id="name" required v-model="product.name" />
    ...
    <button @click="addProduct" class="addProduct">Add Product</button>
  </div>
</template>
```

3. A create product: allows creating a new product, and contains an **add product** button, when pressed on, the product will be added and we will be directed to the **All posts view**.

The View - APost.vue

```
updateProduct() {  
  fetch(`http://localhost:8082/api/products/${this.product.id}`, {  
    method: "PUT", headers: { "Content-Type": "application/json", },  
    body: JSON.stringify(this.product), })  
    .then((response) => {console.log(response.data);  
      this.$router.push("/api/allproducts"); })  
    .catch((e) => {console.log(e); });},  
deletePost() {  
  fetch(`http://localhost:8082/api/products/${this.product.id}`, {  
    method: "DELETE", headers: {"Content-Type": "application/json"}, })  
    .then((response) => {console.log(response.data);  
      this.$router.push("/api/allproducts"); })  
    .catch((e) => {console.log(e); }); }, },  
mounted() { this.fetchAPost(this.$route.params.id); }, },  
</script>
```

2. A single product: fetches and presents a single product based on the passed id. This view also contains **update** and **delete buttons**, when pressed on, the product will be updated/deleted and we will be directed to the **All products view**.

The View - AddProduct.vue

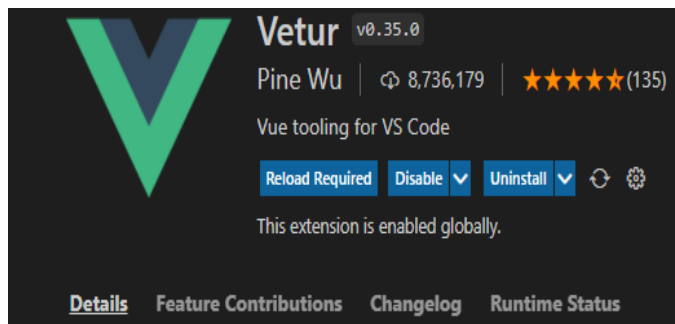
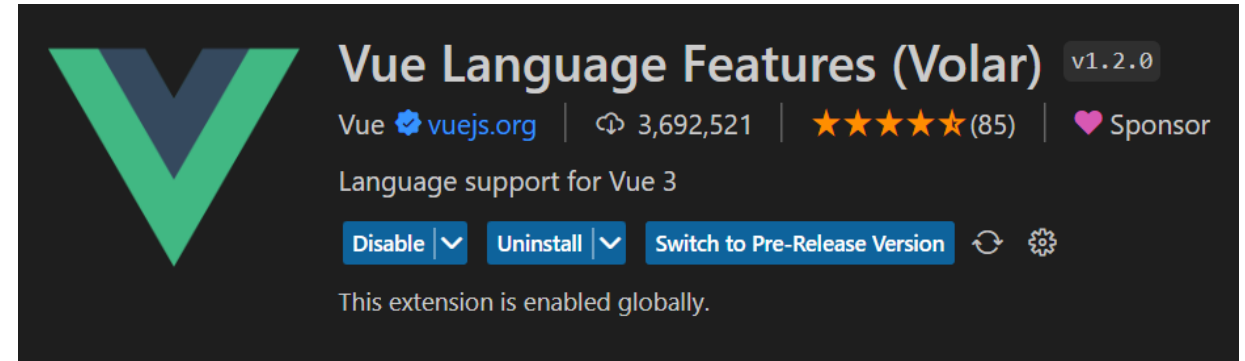
```
<script>
export default {
  name: "AddProduct",
  data() {return { product: {id: "", code: "", name: "", ... }, }; },
  methods: {
    addProduct() {
      var data = {id: this.product.id, code: this.product.code, ..., ...};
      fetch("http://localhost:8082/api/products", {
        method: "POST",
        headers: {"Content-Type": "application/json", },
        body: JSON.stringify(data)})
      .then((response) => {console.log(response.data);
        this.$router.push("/api/allproducts"); })
      .catch((e) => {console.log(e);
        console.log("error");});},},};
</script>
```

3. A create product: allows creating a new product, and contains an **add product** button, when pressed on, the product will be added and we will be directed to the **All posts view**.

Useful Extensions

Useful Extensions

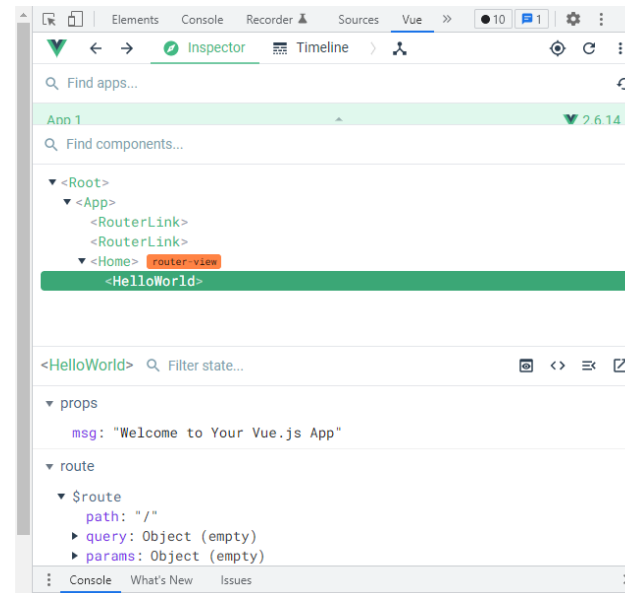
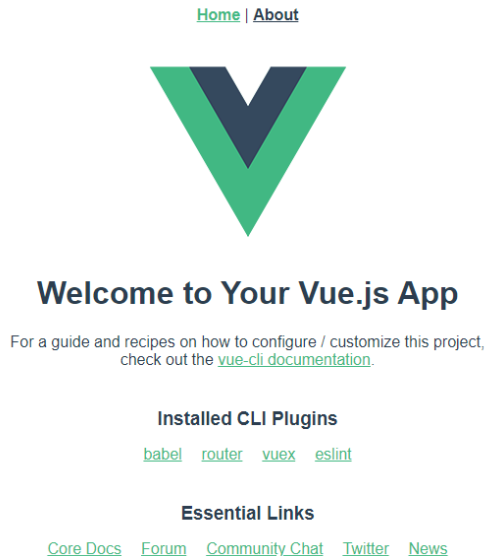
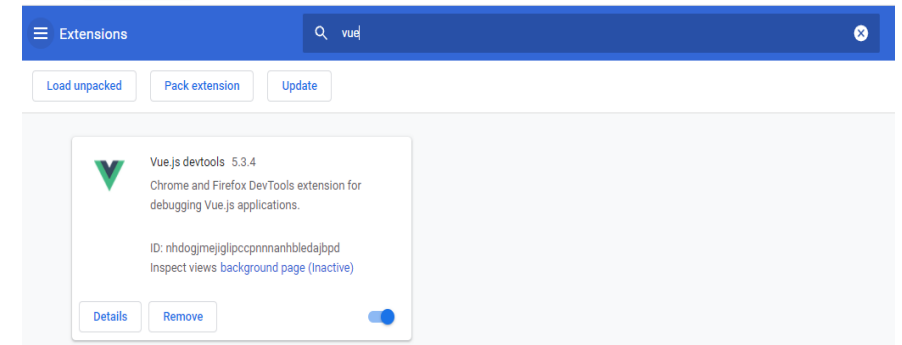
Install the **Volar** extension if you are using **VSCode**, or try to find a similar extension if you are not using **VSCode**. It will be very helpful when coding.



If you were using the **Vetur** extension, you will be suggested to disable it and install Volar.

Useful Extensions

Devtools extension (Google Chrome/Firefox). If installed successfully, its icon should appear among the icons of extensions in your browser.



While on the same page, right-click -> inspect and chose the newly added tab, namely the Vue tab.

Vue.js

Try it yourself ...

Vue.js – basic

[Home](#) | [ViewOne](#) | [FetchAPI](#) | [About](#)

This is a new view page

Props: Hi this is a new compo 1

Data: We are learning Vue

Say Hi

v-on

Hi, I am a v-text

v-html

Hi, I am a v-HTML

v-on

Hello John, you have clicked -1 times

Hello John, you have clicked -1 times

Hello John, you have clicked -1 times

v-show

Hello! I will appear when count > 4

v-once through componenet

Props: Hi this is a new compo - v-once

Data: We are learning Vue

"Appear if random number not bigger than 0.4"

v-bind



Hard coded Image source

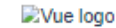


Image source provided without bind



Image source provided with bind

v-model vs v-bind

using v-bind

using v-model

You wrote

v-for, props: and components

This is a list of Componenet

Id: 1

Msg description: msg description 1

Id: 2

Msg description: msg description 2

Id: 3

Msg description: msg description 3

Already available in the shared repo, and the code is annotated to facilitate your understanding, thus, try familiarize yourself with it before the lab.

Vue.js – basic – Simple Fetch API

[Home](#) | [Form Val.](#) | [FetchAPI](#) | [FAQ](#) | [About](#)

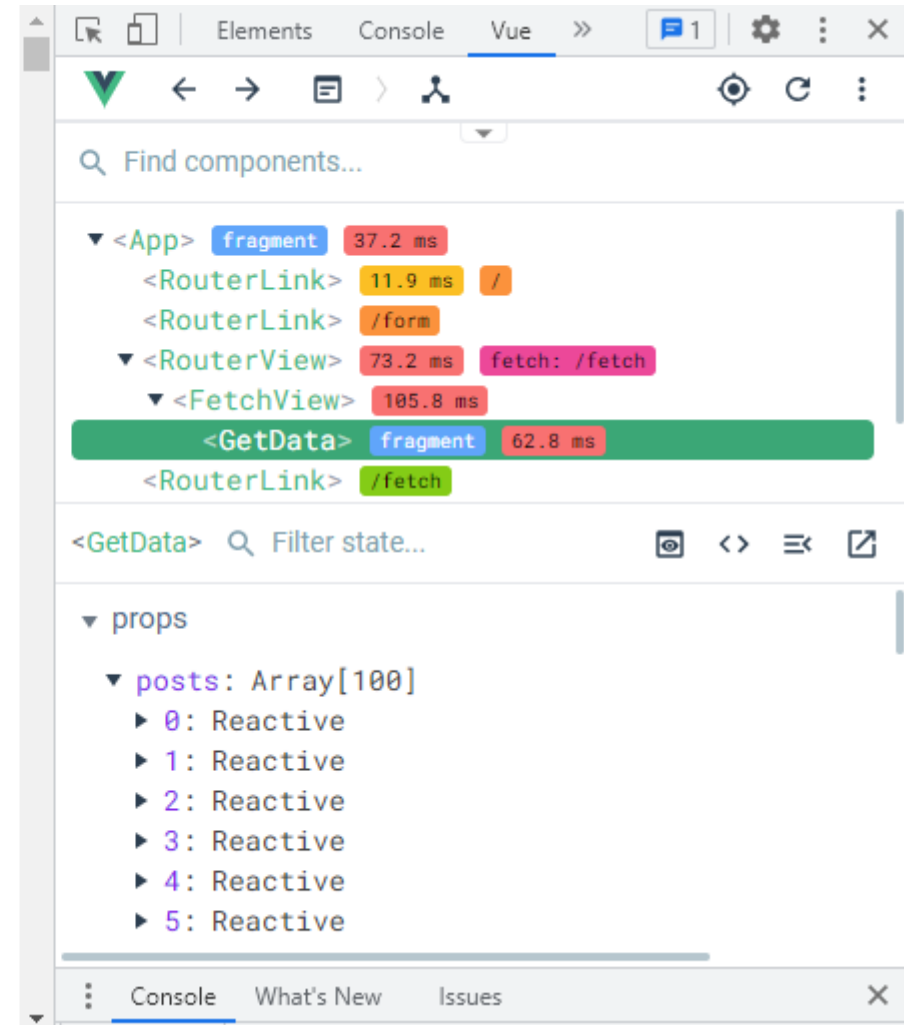
This is FetchAPI page

A simple example of using Fetch API

Title: sunt aut facere repellat provident occaecati excepturi optio reprehenderit

Body: quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto

Title: qui est esse



- A simple example on using Fetch API to fetch and output data in the Vue.js environment.

Extra resources

- **Vue.js – Tutorial** (<https://vuejs.org/tutorial/#step-1>): an easy step by step tutorial about the essentials of Vue.js
- <https://vuejs.org/>



UNIVERSITY OF TARTU

**Thank You
for your attention**

Mohamad Gharib
mohamad.gharib@ut.ee



unitartu



tartuuniversity

